

思维图：用大语言模型求解复杂问题

Maciej Besta^{1*}, Nils Blach^{1*}, Ales Kubicek¹, Robert Gerstenberger¹,
Michał Podstawski², Lukas Gianinazzi¹, Joanna Gajda³, Tomasz Lehmann³,
Hubert Niewiadomski³, Piotr Nyczyk³, Torsten Hoefer¹

¹ETH Zurich, ²Warsaw University of Technology, ³Cledar
bestam@inf.ethz.ch, nils.blach@inf.ethz.ch, htor@inf.ethz.ch

摘要

我们提出 Graph of Thoughts (GoT): 一个超越 Chain-of-Thought、Tree of Thoughts (ToT) 等范式, 增强大语言模型 (LLM) 提示能力的框架。GoT 的核心思想及主要优势, 是将 LLM 生成的信息建模为任意图: 信息单元 (“LLM 思考”) 作为顶点, 边表示顶点间的依赖。这使得任意思考能够组合为相互补充的结果, 从整个思考网络中提炼精华, 或借助反馈循环改进思考。我们在不同任务上展示了 GoT 相对于已有先进方法的优势, 例如排序质量比 ToT 提高 62%, 同时成本降低 >31%。GoT 能扩展新的思考变换, 因而可用于探索新的提示方案。本工作使 LLM 推理更接近人类思考, 以及递归反馈等形成复杂网络的大脑机制。

网站与代码: <https://github.com/spcl/graph-of-thoughts>

1 引言

大语言模型 (LLM) 正成为 AI 的主导力量。近年来, 主要基于仅解码器 Transformer 变体 [65] 的模型迅速发展, 例如 GPT [13, 14, 53, 54]、PaLM [19] 和 LLaMA [63]。

提示工程是一种资源高效的 LLM 任务求解方法: 将任务描述放入发送给 LLM 的输入中; 如果描述设计得当, LLM 就能借助基于 Token 的自回归文本生成机制完成任务。提示可以包含带答案的任务示例, 即少样本提示, 也称上下文学习 (ICL); 也可以完全没有示例, 即零样本提示。近年来的研究表明, 这一机制能解决涉及数学、常识或符号推理的广泛任务。

Chain-of-Thought (CoT) [71] 在提示中除任务输入、输出外, 还包含推理的中间步骤, 即中间 “思考”。CoT 无须更新模型即可显著增强 LLM 求解能力。其重要改进 Self-Consistency with CoT (CoT-SC) [67] 生成多条 CoT, 再选择最佳结果。近期, Tree of Thoughts (ToT) [43, 75, 77] 将 CoT 和 CoT-SC 扩展为以树表示的 LLM 推理过程, 支持不同思考路径, 并提供从不理想结果回溯等新能力。然而, ToT 强制思考过程遵循固定树结构, 仍从根本上限制提示中的推理能力。

我们认为, 让 LLM 思考形成任意图结构, 可以获得更强的提示能力。这受到人类推理、大脑结构和算法执行等现象的启发。人类构思新想法时, 并非只沿一条思考链前进 (如 CoT), 或尝试彼此分离的多条路径 (如 ToT), 而是形成更复杂的思考网络。例如, 先探索一条推理链, 再回溯并启动另一条; 随后发现先前推理链的某个想法可以与当前想法结合, 于是将二者合并成新方案, 保留优点并消除缺点。类似地, 大脑形成复杂网

络, 具有递归反馈等图状模式 [28]。算法执行也呈现网络化模式, 通常以有向无环图表示。将相应的图所支持的变换应用于 LLM 思考, 有望得到更强的提示能力, 而 CoT 或 ToT 难以自然表达这些变换。

我们观察到, 将 LLM 推理过程建模为图, 可以自然支持上述及许多其他思考变换。因此提出 Graph of Thoughts (GoT), 通过网络化推理增强 LLM 能力 (贡献 #1)。在 GoT 中, LLM 思考是顶点, 思考之间的依赖是边。通过构造具有多条入边的顶点, 可以聚合任意思考。GoT 所采用的图抽象, 将 CoT 和 ToT 无缝推广到更复杂的思考模式, 无须任何模型更新。

不过, 实现 GoT 需要解决多项设计挑战。例如, 不同任务应采用什么图结构? 如何聚合思考以最大化准确率、最小化成本? 为回答这些问题, 我们精心设计了实现 GoT 的模块化架构 (贡献 #2), 有两个主要设计特点。首先, 我们能够细粒度控制每个思考, 完全控制与 LLM 的持续交互, 并应用复杂思考变换, 例如将当前推理中最有希望的思考合并为新思考。其次, 架构能够无缝扩展新的思考变换、推理模式 (即思考图) 和 LLM 模型, 使用户借助 GoT 快速验证新的提示思想, 并试验 GPT-3.5、GPT-4 或 Llama-2 [64] 等不同模型。

我们展示 GoT 的多个用例, 包括排序、摘要中的关键词计数、集合运算和文档合并, 并详细说明如何用图范式实现 (贡献 #3)。我们评测 GoT, 展示其相对于先进方法的优势 (贡献 #4)。整体而言, GoT 尤其适合能够自然分解为较小子任务、分别求解后再合并最终结果的任务。例如, 在排序质量上, 它相对于 CoT 和 ToT 分别改善 $\approx 70\%$ 和 $\approx 62\%$, 同时相对于 ToT 降低 >31% 的成本。

我们对 GoT 与其他提示方案作定性比较。¹结果见表 1。GoT 是唯一支持在提示内执行任意图状思考变换 (例如聚合) 的方案, 涵盖了此前提出的方案。

最后, 我们提出评估提示策略的新指标——思考的信息体积 (贡献 #5), 以更好地理解不同方案的差别。对于给定思考 v , v 的信息体积是能够通过有向边到达 v 的 LLM 思考数量。直观上, 它们都是可能对 v 作出贡献的思考。我们表明, GoT 引入聚合等思考变换, 使思考能够具有比其他方案大得多的信息体积。

¹我们不纳入近期称为 Graph-of-Thought 的方案 [79], 因为它并非提示方案。虽然名称暗示与 ToT、CoT 密切相关, 但它通过微调更新模型, 因此不在本工作的研究范围内。类似地, graph-of-thoughts 仓库 [52] 并不支持一般的图推理, 而是采用带 BFS 的 ToT。

*同等贡献

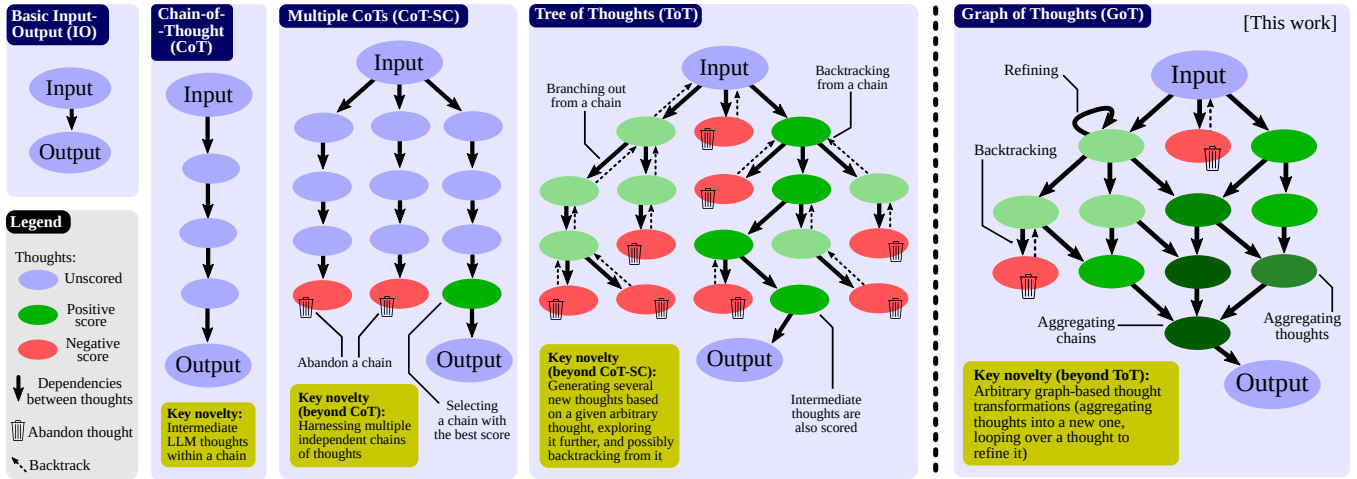


图 1: Graph of Thoughts (GoT) 与其他提示策略的比较。图内英文对照见文末译者补充。

方案	Sc?	Mc?	Tr?	Ag?
Chain-of-Thought (CoT) [71]	■	×	×	×
Self-Consistency with CoT [67]	■	■	×	×
Thought decomposition (思考分解) [75]	■	■	■	×
Tree-of-Thought (ToT) [43]	■	■	■	×
Tree of Thoughts (ToT) [77]	■	■	■	×
Graph of Thoughts (GoT)	■	■	■	■

表 1: 不同提示方案所支持的思考变换比较。Sc?: 单条思维链? Mc?: 多条思维链? Tr?: 思维树? Ag?: 任意思考图? ■: 完全支持; ■: 部分支持; ×: 不支持。

2 背景与符号

首先介绍背景概念和符号。

2.1 语言模型与上下文学习

与 LLM 的对话由用户消息（提示）和 LLM 回复（思考）组成。遵循已有符号 [77]，将参数为 θ 的预训练语言模型 (LM) 记为 p_{θ} 。 $x, y, z, \dots$ 等小写字母表示 LLM 思考。我们有意不统一规定单个“思考”的形式，而让它由用例决定。因此，思考可以是一段文字（如文章摘要）、一份文档（如文档生成）、一个代码块（如代码调试或优化）等。下面介绍具体的提示方法。

输入-输出 (IO) 输入-输出 (IO) 提示是一种直接方法：LLM 将输入序列 x 直接转换为输出 y ，没有中间思考。

思维链 (CoT) 第二种方法 Chain-of-Thought (CoT) 在 x 与 y 之间加入中间思考 $a_1, a_2, \dots$ 。

相对于普通 IO 基线，该策略显著改善多种语言模型任务，例如数学谜题 [71] 或一般数学推理 [24]。

多条 CoT 第三种方法将 CoT 推广为多条 CoT：生成 k 条相互独立的思维链，再根据预先规定的评分指标返回结果最佳的一条。

Wang 等人在称为 Self-Consistency with CoT (CoT-SC) 的方案中引入了这一方法 [67]。它提供探索不同推

理路径的机会，增强了 CoT；但不能在单条路径内进行回溯等“局部探索”。

思维树 (ToT) 最后，Tree of Thoughts (ToT) 由 Yao [77] 和 Long [43] 独立提出，后者称为 Tree-of-Thought；思考分解等其他方案 [75] 也在一定程度上隐含采用了这一思想。它将推理过程建模为思考树，进一步增强 CoT-SC。

单个树节点表示部分解。基于一个节点，思考生成器构建给定数量 k 个新节点；随后，状态评估器为各节点评分。依用例而定，评估可以由 LLM 自身完成，也可以使用人工评分。最后，所用搜索算法（如 BFS 或 DFS）决定树的扩展顺序。

3 GoT 框架

下面详细介绍 GoT 框架。图 1 展示其结构，并与其他提示策略比较。

形式化而言，GoT 可建模为四元组 $(G, \mathcal{T}, \mathcal{E}, \mathcal{R})$ ，其中 G 为“LLM 推理过程”（上下文内全部思考及其关系）， $\mathcal{T}$ 为可能的思考变换， $\mathcal{E}$ 为思考评分函数， $\mathcal{R}$ 为选择最相关思考的排序函数。

3.1 推理过程

将推理过程建模为有向图 $G = (V, E)$ ，其中 V 为顶点集， $E \subseteq V \times V$ 为边集。 G 是有向图，因此边属于有序顶点对的子集 $E \subseteq V \times V$ 。

顶点包含当前问题的一个解，可以是初始解、中间解或最终解。思考的具体形式取决于用例，例如写作中的段落、排序中的数字序列。有向边 (t_1, t_2) 表示思考 t_2 使用 t_1 作为“直接输入”构建，即显式指示 LLM 用 t_1 生成 t_2 。

某些用例中的节点属于不同类别。例如，写作任务的一些顶点表示写段落的计划，另一些表示实际文本段落。此时，GoT 使用异构图 $G = (V, E, c)$ 建模推理，其中 c 将顶点 V 映射到各自类别 C ；上述例子中， $C = \{plan, par\}$ 。因此，任一顶点 v 均可表示不同的推理方面。

将 G 与 LLM 推理过程对应后，通过对 G 应用思考变换推进推理。例如，将目前评分最佳的思考合并为新

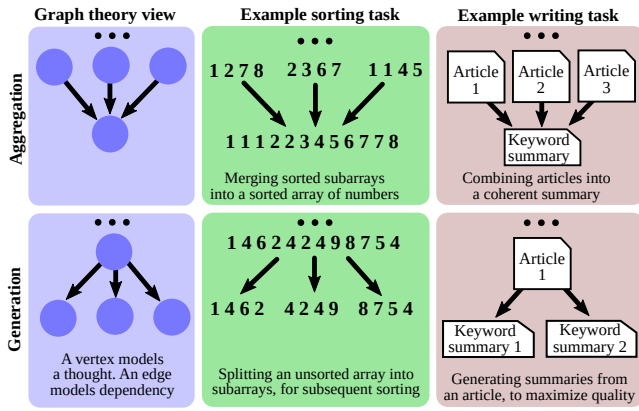


图 2: 聚合与生成两类思考变换的示例。图内英文对照见文末译者补充。

思考，或对一个思考反复迭代以改进它。这些变换严格扩展了 CoT、CoT-SC 或 ToT 所支持的变换集合。

3.2 思考变换

GoT 的图推理模型支持新的思考变换，我们称为**图所支持的变换**。例如，写作时将多篇输入文章合为连贯摘要；排序时将多个已排序数字子数组组合为最终有序数组。图 2 展示聚合与生成的例子。

形式化而言，每项变换可建模为 $\mathcal{T}(G, p_\theta)$ ，其中 $G = (V, E)$ 是反映当前推理状态的图， p_θ 为所用 LLM。 $\mathcal{T}$ 通常通过添加新顶点及其入边修改 G 。有 $G' = \mathcal{T}(G, p_\theta) = (V', E')$ ，其中 $V' = (V \cup V^+) \setminus V^-$ ， $E' = (E \cup E^+) \setminus E^-$ 。 V^+ 与 E^+ 分别是插入 G 的新顶点与新边，表示新思考及其依赖。为最大化 GoT 的表达能力，我们也允许用户显式删除思考，指定需要删除的顶点、边，分别为 V^- 与 E^- 。用户负责确保集合 V^+, E^+, V^- 和 E^- 所定义的变换一致，例如不能删除不存在的顶点。这使得在节省上下文空间时，可以无缝纳入删除无望推理部分的方案。 $\mathcal{T}$ 的具体形式及其对 G 的影响由变换决定。我们先介绍主要的图式思考变换，再说明 GoT 如何涵盖早期方案的变换。除非另有说明， $V^- = E^- = \emptyset$ 。

聚合变换 首先，GoT 可以将**任意思考聚合**成新思考，组合并强化各自优点，同时消除缺点。

在只创建一个新顶点的基本形式中， $V^+ = \{v^+\}$ ， $E^+ = \{(v_1, v^+), \dots, (v_k, v^+)\}$ ，其中 $v_1, \dots, v_k$ 为合并的 k 个思考。更一般地，它不只聚合单个思考，还能**聚合推理路径**，即较长的思维链。只需从表示各链最终思考的顶点 $v_1, \dots, v_k$ 添加出边，连接到合并这些链的单个思考 v^+ ，即可实现。

改进变换 另一类变换通过修改内容，**改进**当前思考 v ： $V^+ = \{v\}$ ， $E^+ = \{(v, v)\}$ 。图中的自环表示对思考进行迭代，其连接关系与原思考相同。

生成变换 最后，可以基于单个已有思考 v **生成一个或多个新思考**。这一类涵盖 ToT、CoT-SC 等早期方案中的类似推理步骤。形式化而言， $V^+ = \{v_1^+, \dots, v_k^+\}$ ， $E^+ = \{(v, v_1^+), \dots, (v, v_k^+)\}$ 。

3.3 思考的评分与排序

通过评分判断当前解是否足够好。分数建模为一般函数 $\mathcal{E}(v, G, p_\theta)$ ，其中 v 为待评估思考。为了保持最大一般性， $\mathcal{E}$ 使用整个推理过程的状态 (G)，因为某些评估场景中的分数相对于其他思考而定。

GoT 也可以对思考排序，建模为函数 $\mathcal{R}(G, p_\theta, h)$ 。其中， h 指定 $\mathcal{R}$ 应从 G 中返回多少个排名最高的思考。 $\mathcal{R}$ 的具体形式取决于用例，但我们通常采用简单有效的策略：返回分数最高的 h 个思考，即 $v_1, \dots, v_h = \mathcal{R}(G, p_\theta, h)$ 。

$\mathcal{E}$ 与 $\mathcal{R}$ 的具体形式取决于用例，第 5 节介绍细节。例如，排序任务的分数或排名对应正确排序的元素数；使用误差作为分数时，则对应错误元素数。

4 系统架构与可扩展性

GoT 架构包含一组相互作用的模块，见图 3 **蓝色区域**：提示构造器（准备发给 LLM 的消息）、解析器（从 LLM 思考中提取信息）、评分模块（验证并评分 LLM 思考），以及控制器（协调整个推理过程、决定如何推进）。控制器还包含两个重要组成：操作图（Graph of Operations, GoO）与图推理状态（Graph Reasoning State, GRS）。GoO 是静态结构，规定给定任务的图分解，即要对思考应用哪些变换，以及变换顺序和依赖关系。GRS 是动态结构，保存持续推理的状态，包括思考历史及各思考状态。

4.1 提示构造器

提示构造器准备发送给 LLM 的提示，负责在提示中编码图结构的具体方式。GoT 架构提供对图结构的完整访问，使用户能够实现针对具体用例的图编码。

4.2 解析器

解析器从 LLM 思考中提取信息，为每个思考构建包含所提取信息的思考状态，再据此更新 GRS。

4.3 评分与验证

首先验证某个思考是否满足可能的正确性条件，再为它评分。视评分方式而定，模块可以调用 LLM，也可以使用人工评分。排序等用例则采用简单的本地评分函数。

4.4 控制器

控制器实现从 GRS 中选择思考的具体策略，也决定对哪些思考应用哪些变换，再把信息交给提示构造器。它还判断整个过程是否应结束，或是否应启动下一轮 LLM 交互。当前设计中，这由 GoO 指定的执行计划决定。

4.5 GoO 与 GRS

用户构建 GoO 实例，规定思考操作的执行计划。GoO 是静态结构，只在执行开始前构建一次；每个操作对象知道自身的前驱与后继。运行时，GRS 实例持续保存关于 LLM 推理过程的最新信息，包括已执行操作、所有生成思考的状态、有效性与分数，以及其他相关信息。

上述组成提供可扩展的 **API**，便于实现不同提示方案。图 3 的**绿色区域**概述 API，文档提供详细说明；同一图 3 的**红色区域**还给出操作使用的**提示示例**及相应 GRS。

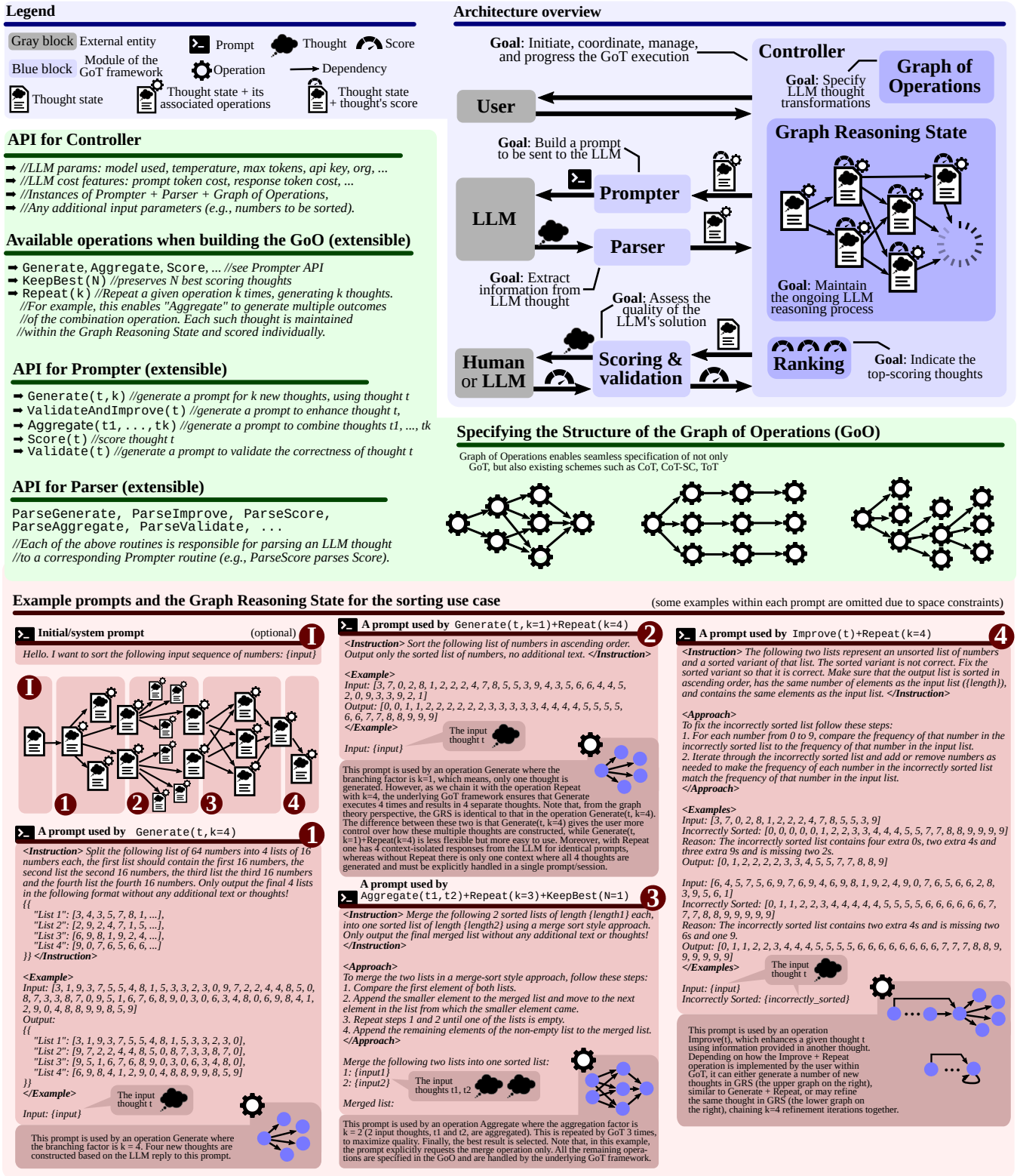


图 3: GoT 的系统架构及各模块 API。用户可以直接扩展设计, 构建新提示方案、试验新思考变换, 并接入不同 LLM。蓝色区域为架构概览, 绿色区域列出 API, 红色区域包含提示示例、相应 GRS 及所用操作。图内英文对照见文末译者补充。

5 用例示例

下面介绍 GoT 的多个用例，其中详细讲解排序，简述其余用例。

5.1 排序

我们着重介绍排序任务的分解及操作图；二者是通过 GoT 实现和执行工作负载的核心。考虑对包含重复值的 0-9 数字排序。当序列超过一定长度时，所考察 LLM 无法稳定地正确排序，因为重复数字的数量会不一致。在 GoT 中采用归并式排序：先把输入数字序列分解为子数组，分别排序，再合并成最终结果。图 4 展示用例及其图分解；一个 LLM 思考就是一个已排序数字序列。

为结果评分，将输入序列记为 $[a_1, a_2, \dots, a_n]$ ，输出序列记为 $[b_1, b_2, \dots, b_m]$ 。用以下分数衡量“错误范围”：

$$\text{error-scope} = X + Y$$

其中， $p \in \{1, \dots, m\}$ ， $q \in \{1, \dots, n\}$ ，且

$$X = \sum_{i=1}^{m-1} \text{sgn}(\max(b_i - b_{i+1}, 0)),$$

$$Y = \sum_{i=0}^9 |\{b_p : b_p = i\}| - |\{a_q : a_q = i\}|$$

X 表示有多少相邻数字对的顺序错误。如果位置 i 和 $i+1$ 的数字顺序错误，即 $b_i > b_{i+1}$ ，求和中的表达式返回 1，错误分数增加 1；若顺序正确，则为 0。 Y 衡量输出序列保留各数字频次的程度。对于每个数字 x ($x \in \{0, \dots, 9\}$)，计算输入中等于 x 的元素数量与输出中等于 x 的元素数量之差。若输出完整保留 x 的频次，差为 0；数量每偏差一次，“错误范围”增加 1。最后对全部 x 求和。绘制该分数时，为提高图的清晰度，还采用截断 $\min(\text{error-scope}, n)$ ，因为 IO、CoT 等基线会产生许多错误范围较高的异常值。如果希望使用描述“正确排序范围”的正向分数，可以取 $\max(n - \text{error-scope}, 0)$ 。

5.2 集合运算

我们还考虑集合运算，重点是集合交集。集合运算，特别是求交集，广泛用于基因组比较、文档比较、模式匹配等问题 [9-11, 20, 27, 38, 50, 58]。

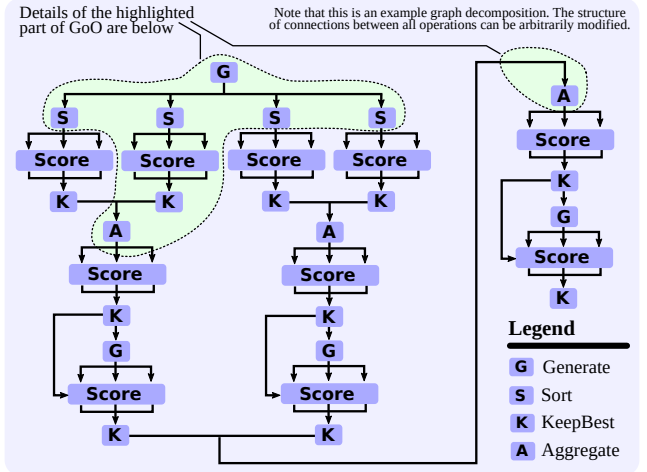
两个集合的求交实现类似排序：将第二个输入集合拆为子集，借助 LLM 分别计算各子集与第一个输入集合的交集，再聚合各部分交集得到最终结果。评测采用 32、64 和 128 个元素的集合，并将两集合共有元素比例设为 25%-75%。

分数表示最终交集中遗漏或错误纳入的元素总数。令两个输入集合分别为 $A = [a_1, a_2, \dots, a_n]$ 、 $B = [b_1, b_2, \dots, b_m]$ ，输出集合为 $C = [c_1, c_2, \dots, c_m]$ ，则

$$\text{error-scope} = X_1 + X_2 + X_d$$

其中， $X_1 = |C \setminus (A \cap B)|$ 是 C 中不应出现的元素数， $X_2 = |(A \cap B) \setminus C|$ 是 C 遗漏的元素数， X_d 为 C 中的重复元素数，因为 LLM 用自然语言中的列表来表达集合。若希望使用表示“正确计算范围”的正向分数，可以取 $\max(n - \text{error-scope}, 0)$ 。

Graph of Operations (GoO) for sorting 64 numbers



Details of the highlighted part of the GoO from above

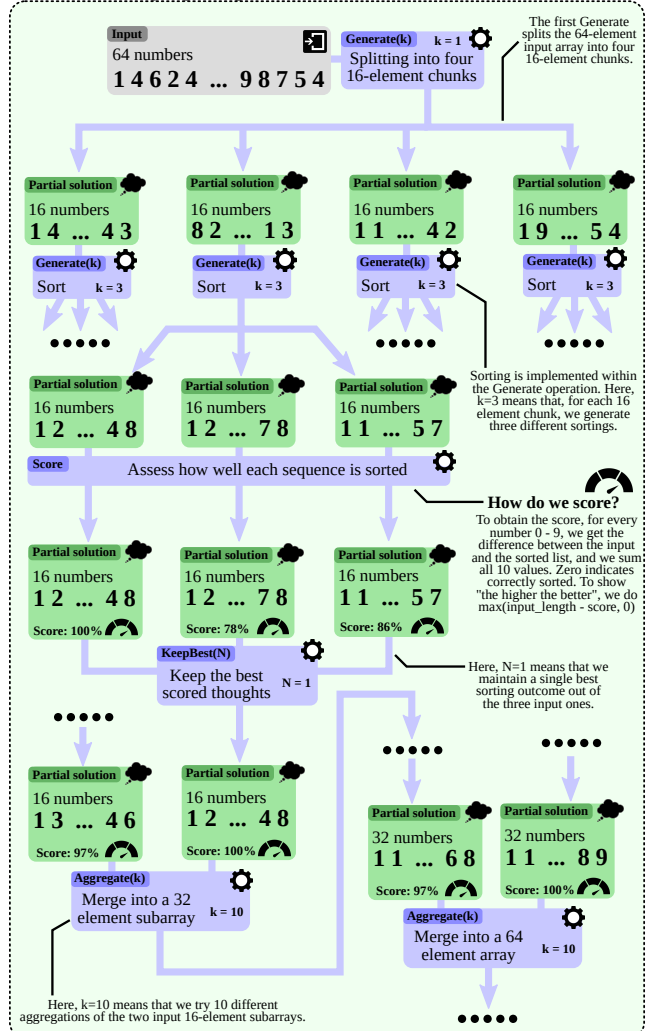


图 4: GoT 排序用例的图分解示例。所用全部操作 (Generate、Aggregate、Score、KeepBest) 见图 3。图内英文对照见文末译者补充。

5.3 关键词计数

关键词计数统计输入文本中属于某一类别的关键词频次；本实现以国家为类别。GoT 将输入文本分成多个片段，统计各片段的关键词，再聚合部分结果。片段数可以配置，也可交由 LLM 决定，因此甚至可以把每个句子当作单独片段。评分时，先对每个关键词求预测计数与正确计数的绝对差，再将全部差值相加，得到最终分数。

5.4 文档合并

最后，我们还提供文档合并。目标是基于多份内容部分重叠的输入保密协议 (Non-Disclosure Agreement, NDA)，生成一份新协议，尽量减少重复、最大限度保留信息。文档合并适用于多种场景，例如法律程序中将多个信息来源合为一份文档或文章。为了评分，向 LLM 查询两个值；每个值查询三次取平均。第一项表示重复程度：10 表示没有重复，0 表示至少一半信息重复；第二项表示信息保留程度：10 表示全部保留，0 表示完全没有保留。最终取两项值的调和平均数。

6 时延与信息体积的权衡

下面展示 GoT 如何改善早期提示方案在时延（沿思考图到达给定最终思考所需的跳数）与信息体积之间的权衡。对于给定思考 t ，其信息体积定义为可能影响 t 的前序 LLM 思考数量。形式化而言， t 的信息体积是思考图中存在通向 t 的路径的思考数量。假设生成单个思考耗时 $O(1)$ ，且各提示方案的总成本固定为 $\Theta(n)$ 。

各方案的结构如下：CoT-SC 是从同一初始思考出发的 k 条独立链；ToT 是一棵完全 k 叉树；GoT 则将一棵完全 k 叉树的叶节点，与相同规模但边方向反转的“镜像” k 叉树连接。

分析详见表 2。CoT 的信息体积可以达到 N ，但代价是较高的时延 N 。CoT-SC 按因子 k （对应其分支因子）降低时延，但信息体积也同时缩小 k 倍。ToT 的时延为 $\log_k N$ ，信息体积仍较低。GoT 是唯一同时具有较低时延 $\log_k N$ 和较高信息体积 N 的方案。这是因为它使用思考聚合，使图分解中任一中间思考都能够到达最终思考。

方案	时延	信息体积
Chain-of-Thought (CoT)	N	N
Self-Consistency with CoT (CoT-SC)	N/k	N/k
Tree of Thoughts (ToT)	$\log_k N$	$O(\log_k N)$
Graph of Thoughts (GoT)	$\log_k N$	N

表 2: 不同提示方案在时延与信息体积之间的基本权衡。GoT 提供最佳权衡。

7 评测

我们展示 GoT 相对于先进方法的优势。主要比较 ToT，因为已有结果表明它稳定优于其他方案；为了广泛比较，也试验 IO、CoT 和 CoT-SC。分析涉及较大的评测空间，因此只给出具有代表性的结果，省略不能带来相关新认识的数据，例如 CoT-SC。

7.1 评测方法

每个任务和比较基线使用 100 个输入样本。除非另有说明，温度设为 1.0、上下文大小为 4k。每次实验固定各方案的思考数量，使实验成本相近。

参数。我们广泛试验分支因子 k 与层数 L ，确保 GoT 的比较对象配置具有成本效率和良好表现。绘制两种 ToT 变体：一种 k 较高、深度较低 (ToT)，另一种 k 较低、 L 较高 (ToT2)。

通常在较稀疏的生成轮次（较低 k ）与更多轮次（较大 L ）之间寻找更佳权衡。每轮更多回复一般会更昂贵，例如图 7 中总回复数为 80 与 60，成本却为 \$6 与 \$3。我们也尝试不同问题规模 P ；排序中， P 表示待排序数字数量。

所用 LLM。由于预算限制，主要使用 GPT-3.5。我们也试验了 Llama-2，但它通常比 GPT-3.5 效果差、运行慢，难以获得足够样本。

7.2 GoT 优势分析

结果分别见图 5（排序）、图 6（集合交集）、图 7（关键词计数）和图 8（文档合并）。具体用例见第 5 节。总体而言，GoT 相对于所有考察基线提高了结果质量，并相对于 ToT 降低了推理成本。

GoT 与 ToT。对于全部问题实例，GoT 都显著优于 ToT 和 ToT2。ToT 通常比 ToT2 质量略高，但成本也高得多。GoT 的成本始终低于 ToT，与 ToT2 相近，有时较低、有时较高。例如，排序规模为 $P = 128$ 时，GoT 相对于 ToT 将错误数中位数降低 $\approx 62\%$ ，提高排序质量，同时成本降低 $>31\%$ 。这些优势源于它能将复杂任务分解为简单子任务，独立求解后逐步合并为最终结果。

GoT 与 IO、CoT。GoT 的结果质量始终远高于 IO/CoT。例如，排序规模为 $P = 64$ 时，相对于 CoT 和 IO，GoT 的错误数中位数分别降低 $\approx 65\%$ 和 $\approx 83\%$ 。但 GoT 与 ToT 的成本远高于 IO 和 CoT。主要原因是，我们配置 CoT 时，如果延长推理链不能改善结果，就不会人为增加链长。GoT 和 ToT 成本较高，是因为每次 Generate 操作生成 k 个新思考；多思考也是 GoT 质量较高的原因之一。

所处理问题的复杂度增加。更关键的是，GoT 相对于全部基线的质量优势随问题规模 P 增大而增加。例如排序中， $P = 32$ 时相对于 ToT2 只略有改善； $P = 64$ 时错误数中位数降低 $\approx 61\%$ ， $P = 128$ 时降低 $\approx 69\%$ ，四分位数也相应改善。其他方案的结果符合直觉，例如 IO 随 P 增大持续变差，因为单个思考不太可能解决大型问题实例。整体上，分析表明 GoT 适合复杂问题，因为问题规模增大时，执行安排通常也更复杂。

7.3 关于任务分解的讨论

将任务拆成子任务分别求解时，输入和回复的 Token 数通常随分解程度成比例减少。但提示的“静态”部分，即少样本示例，可能变成显著开销，见图 7 的 GoT4 到 GoT8。我们观察到，少样本示例通常也能缩小：例如展示关键词计数的文本片段可以更短，同时仍能反映实际输入规模，从而主动降低成本，见图 7 中 GoT8 与 GoTx 的差异。图分解的总体目标，是将任务分到 LLM 大多数时候仅凭一次提示、或少量额外改进步骤就能正确求解的程度。这显著减少后期图探索所需的改进或细化步骤。此外，结果表明，合并或拼接子结果通常比从

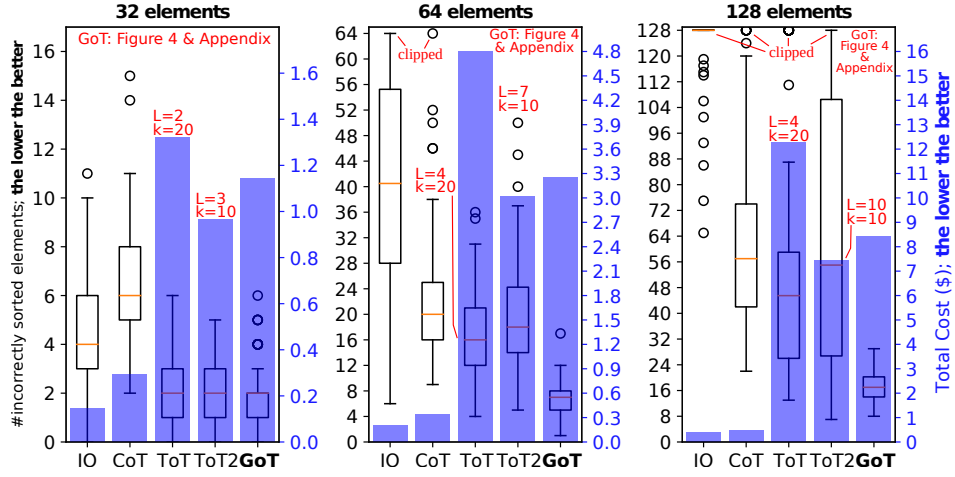


图 5: ChatGPT-3.5 排序任务的错误数与成本。L 和 k 表示 ToT 的结构，见第 3.2 节与第 6 节。图内英文对照见文末译者补充。

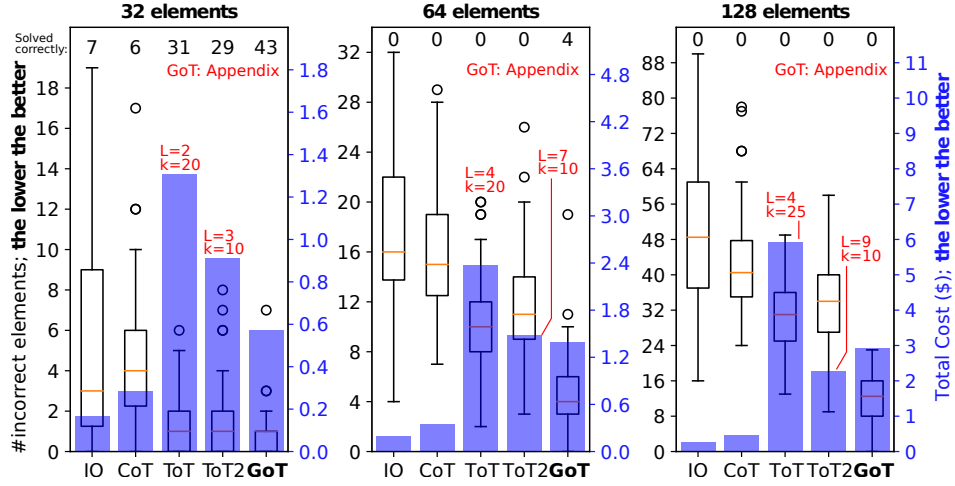


图 6: ChatGPT-3.5 集合交集任务的错误数与成本。L 和 k 表示 ToT 的结构，见第 3.2 节与第 6 节。图内英文对照见文末译者补充。

头求解大实例更简单，因此 LLM 往往能够成功聚合最终解。

8 相关工作

我们概述 GoT 与相关工作的关系。

8.1 提示范式与方法

第 1 节及表 1 已详细介绍不同提示范式。

还有许多与提示相关的工作，下面简述最相关者；更广泛的介绍可见专门综述 [34, 40, 69, 70]。Wang 等人提出 Plan-and-Solve，用显式规划阶段增强 CoT [66]；Fu 等人利用复杂度标准改进 CoT 内的提示 [29, 67]。自学推理器 STaR [80] 生成多条思维链，选择有效者。类似地，Shum 等人 [61] 生成 CoT 候选池，根据候选是否符合标准答案，以及基于策略梯度的方法选择最佳候选。自动生成提示可以解决 CoT 的扩展问题 [41, 42, 59]。Zhou 等人提出从候选集合选择最佳提示 [84]。Skeleton-

of-Thought [47] 先生成若干骨架答案，即每项 3-5 个词的简短要点，再于第二步并行展开各要点。

最后，提示链将不同 LLM 级联，通过不同上下文提示不同模型，提供更强大推理 [21, 23, 48, 51, 72, 73, 73]。GoT 关注单个上下文中的能力，因此与这类方案可以配合。

8.2 自我反思与自我评估

近期提出了自我反思和自我评估 [45, 49, 60, 75, 85]，用于改进不同任务，例如代码生成 [17] 或计算机操作任务 [39]。GoT 在决定如何扩展提示内的思考图时，部分依赖自我评估。

8.3 LLM 与规划

近期许多工作研究如何用 LLM 规划复杂任务 [36, 37, 68, 76, 78, 81]。GoT 可以视为通用框架，通过提供生成复杂图式计划的范式，潜在地增强这些方案。

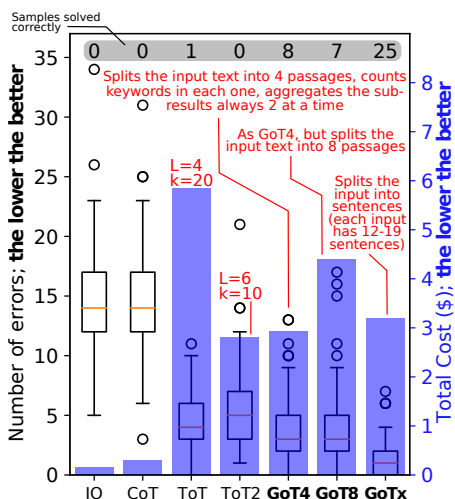


图 7: ChatGPT-3.5 关键词计数的错误数与成本。L 和 k 表示 ToT 的结构，见第 3.2 节与第 6 节。图内英文对照见文末译者补充。

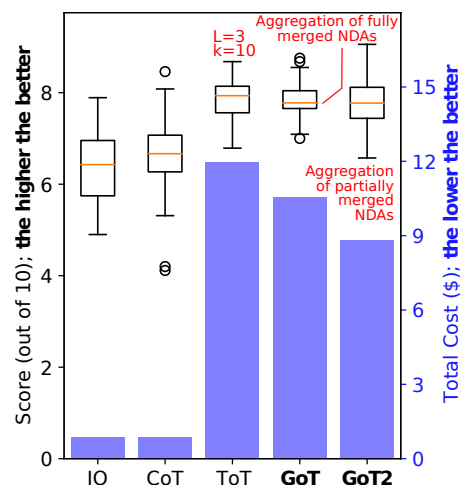


图 8: ChatGPT-3.5 文档合并的分数与成本。L 和 k 表示 ToT 的结构，见第 3.2 节与第 6 节。样本数为 50，上下文大小为 16k Token。图内英文对照见文末译者补充。

8.4 图与图计算

图已成为通用计算领域中极受关注且重要的组成 [31, 32, 44, 46, 56]。近年来，图数据库 [2–4, 7, 55]、图模式匹配 [8, 10, 11, 18, 25, 62]、图流处理 [1, 22, 26]、图机器学习与图神经网络 [5, 6, 12, 16, 30, 33, 33, 57, 74, 82, 83] 等领域受到越来越多关注。图抽象推动了许多现代研究领域：社会科学（研究人类交互）、生物信息学（分析蛋白质结构）、化学（设计化合物）、医学（药物发现）、网络安全（识别入侵机器）、医疗服务（发现提交欺诈理赔的人群）、Web 图分析（提供准确搜索）、娱乐服务（预测电影受欢迎程度）、语言学（建模词间关系）、交通运输（寻找高效路线）、物理学（理解相变及临界现象）等 [15, 20, 35, 38, 44]。

本工作将图抽象用作增强 LLM 提示能力的关键机制。

9 结论

提示工程是大语言模型 (LLM) 研究的重要新领域之一，使用户无须更新模型即可高效使用 LLM；但设计有效提示十分困难。我们提出 Graph of Thoughts (GoT) 新范式，让 LLM 无须更新模型即可有效解决不同任务。核心思想是将 LLM 推理建模为任意图：思考是顶点，思考间的依赖是边。这支持聚合等新思考变换。人类解决任务往往不是线性的，而会将中间解合并成最终解，或发现新认识后改变推理流程。GoT 的图结构反映了这一点。

GoT 优于其他提示方案，例如排序质量比 ToT 提高 62%，同时成本降低 >31%。我们还提出提示方案的新指标——思考的信息体积，用来表示给定 LLM 输出可能携带的信息范围，GoT 在这一指标上同样表现突出。这是走向更有原则的提示工程的一步。过去数十年，图抽象是计算和 AI 多项成功设计的基础，例如用于蛋白质预测的 AlphaFold。本工作将其用于提示工程。

致谢

感谢 Hussein Harake、Colin McMurtrie、Mark Klein、Angelo Mangili 及整个 CSCS 团队，提供 Ault 和 Daint 机器访问权限与出色技术支持。感谢 Timo Schneider 协助 SPCL 的基础设施。

本项目获得 European Research Council 资助 (PSAP 项目，编号 101002047)，以及 European High-Performance Computing Joint Undertaking (JU) 资助，资助协议编号 955513 (MAELSTROM)。本项目得到 ETH Future Computing Laboratory (EFCL) 支持，其经费来自 Huawei Technologies 的捐赠。本项目还获得 European Union 的 HE 研究与创新计划资助，协议编号 101070141 (GLACIATION 项目)。

参考文献

- [1] Besta, M.; Fischer, M.; Kalavri, V.; Kapralov, M.; and Hoefer, T. 2023. Practice of Streaming Processing of Dynamic Graphs: Concepts, Models, and Systems. *IEEE Transactions on Parallel and Distributed Systems*, 34(6): 1860–1876.
- [2] Besta, M.; Gerstenberger, R.; Blach, N.; Fischer, M.; and Hoefer, T. 2023. GDI: A Graph Database Interface Standard. <https://github.com/spcl/GDI-RMA>. Accessed: 2023-09-05.
- [3] Besta, M.; Gerstenberger, R.; Fischer, M.; Podstawski, M.; Blach, N.; Egeli, B.; Mitenkov, G.; Chlapek, W.; Michalewicz, M.; Niewiadomski, H.; Müller, J.; and Hoefer, T. 2023. The Graph Database Interface: Scaling Online Transactional and Analytical Graph Workloads to Hundreds of Thousands of Cores. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '23*. ACM.
- [4] Besta, M.; Gerstenberger, R.; Peter, E.; Fischer, M.; Podstawski, M.; Barthels, C.; Alonso, G.; and Hoefer, T. 2023. Demystifying Graph Databases: Analysis and Taxonomy of Data Organization,

- System Designs, and Graph Queries. *ACM Comput. Surv.*, 56(2).
- [5] Besta, M.; Grob, R.; Miglioli, C.; Bernold, N.; Kwaśniewski, G.; Gjini, G.; Kanakagiri, R.; Ashkboos, S.; Gianinazzi, L.; Dryden, N.; and Hoefer, T. 2022. Motif Prediction with Graph Neural Networks. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '22, 35–45.
 - [6] Besta, M.; and Hoefer, T. 2022. Parallel and Distributed Graph Neural Networks: An In-Depth Concurrency Analysis. arXiv:2205.09702.
 - [7] Besta, M.; Iff, P.; Scheidl, F.; Osawa, K.; Dryden, N.; Podstawski, M.; Chen, T.; and Hoefer, T. 2022. Neural Graph Databases. In *Proceedings of the First Learning on Graphs Conference*, volume 198 of *Proceedings of Machine Learning Research*, 31:1–31:38. PMLR.
 - [8] Besta, M.; Kanakagiri, R.; Kwaśniewski, G.; Ausavarungrun, R.; Beránek, J.; Kanellopoulos, K.; Janda, K.; Vonarburg-Shmaria, Z.; Gianinazzi, L.; Stefan, I.; Luna, J. G.; Golinowski, J.; Copik, M.; Kapp-Schwoerer, L.; Di Girolamo, S.; Blach, N.; Konieczny, M.; Mutlu, O.; and Hoefer, T. 2021. SISA: Set-Centric Instruction Set Architecture for Graph Mining on Processing-in-Memory Systems. In *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO '21, 282–297.
 - [9] Besta, M.; Kanakagiri, R.; Mustafa, H.; Karasikov, M.; Rättsch, G.; Hoefer, T.; and Solomonik, E. 2020. Communication-Efficient Jaccard Similarity for High-Performance Distributed Genome Comparisons. In *Proceedings of the IEEE International Parallel and Distributed Processing Symposium*, IPDPS '20, 1122–1132.
 - [10] Besta, M.; Miglioli, C.; Labini, P. S.; Têtek, J.; Iff, P.; Kanakagiri, R.; Ashkboos, S.; Janda, K.; Podstawski, M.; Kwaśniewski, G.; Gleinig, N.; Vella, F.; Mutlu, O.; and Hoefer, T. 2022. ProbGraph: High-Performance and High-Accuracy Graph Mining with Probabilistic Set Representations. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, SC '22. IEEE.
 - [11] Besta, M.; Vonarburg-Shmaria, Z.; Schaffner, Y.; Schwarz, L.; Kwaśniewski, G.; Gianinazzi, L.; Beránek, J.; Janda, K.; Holenstein, T.; Leisinger, S.; Tatkowski, P.; Ozdemir, E.; Balla, A.; Copik, M.; Lindenberger, P.; Konieczny, M.; Mutlu, O.; and Hoefer, T. 2021. GraphMineSuite: Enabling High-Performance and Programmable Graph Mining Algorithms with Set Algebra. *Proc. VLDB Endow.*, 14(11): 1922–1935.
 - [12] Bronstein, M. M.; Bruna, J.; LeCun, Y.; Szlam, A.; and Vandergheynst, P. 2017. Geometric Deep Learning: Going beyond Euclidean data. *IEEE Signal Processing Magazine*, 34(4): 18–42.
 - [13] Brown, T.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J. D.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; Agarwal, S.; Herbert-Voss, A.; Krueger, G.; Henighan, T.; Child, R.; Ramesh, A.; Ziegler, D.; Wu, J.; Winter, C.; Hesse, C.; Chen, M.; Sigler, E.; Litwin, M.; Gray, S.; Chess, B.; Clark, J.; Berner, C.; McCandlish, S.; Radford, A.; Sutskever, I.; and Amodei, D. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems (NeurIPS '20)*, volume 33, 1877–1901. Curran Associates.
 - [14] Bubeck, S.; Chandrasekaran, V.; Eldan, R.; Gehrke, J.; Horvitz, E.; Kamar, E.; Lee, P.; Lee, Y. T.; Li, Y.; Lundberg, S.; Nori, H.; Palangi, H.; Ribeiro, M. T.; and Zhang, Y. 2023. Sparks of Artificial General Intelligence: Early experiments with GPT-4. arXiv:2303.12712.
 - [15] Chakrabarti, D.; and Faloutsos, C. 2006. Graph Mining: Laws, Generators, and Algorithms. *ACM Comput. Surv.*, 38(1).
 - [16] Chami, I.; Abu-El-Haija, S.; Perozzi, B.; Ré, C.; and Murphy, K. 2020. Machine Learning on Graphs: A Model and Comprehensive Taxonomy. arXiv:2005.03675.
 - [17] Chen, X.; Lin, M.; Schärli, N.; and Zhou, D. 2023. Teaching Large Language Models to Self-Debug. arXiv:2304.05128.
 - [18] Cheng, J.; Yu, J. X.; Ding, B.; Philip, S. Y.; and Wang, H. 2008. Fast Graph Pattern Matching. In *Proceedings of the IEEE 24th International Conference on Data Engineering*, ICDE '08, 913–922.
 - [19] Chowdhery, A.; Narang, S.; Devlin, J.; Bosma, M.; Mishra, G.; Roberts, A.; Barham, P.; Chung, H. W.; Sutton, C.; Gehrmann, S.; Schuh, P.; Shi, K.; Tsvyashchenko, S.; Maynez, J.; Rao, A.; Barnes, P.; Tay, Y.; Shazeer, N.; Prabhakaran, V.; Reif, E.; Du, N.; Hutchinson, B.; Pope, R.; Bradbury, J.; Austin, J.; Isard, M.; Gur-Ari, G.; Yin, P.; Duke, T.; Levskaya, A.; Ghemawat, S.; Dev, S.; Michalewski, H.; Garcia, X.; Misra, V.; Robinson, K.; Fedus, L.; Zhou, D.; Ippolito, D.; Luan, D.; Lim, H.; Zoph, B.; Spiridonov, A.; Sepassi, R.; Dohan, D.; Agrawal, S.; Omernick, M.; Dai, A. M.; Pillai, T. S.; Pellat, M.; Lewkowycz, A.; Moreira, E.; Child, R.; Polozov, O.; Lee, K.; Zhou, Z.; Wang, X.; Saeta, B.; Diaz, M.; Firat, O.; Catasta, M.; Wei, J.; Meier-Hellstern, K.; Eck, D.; Dean, J.; Petrov, S.; and Fiedel, N. 2022. PaLM: Scaling Language Modeling with Pathways. arXiv:2204.02311.
 - [20] Cook, D. J.; and Holder, L. B., eds. 2006. *Mining Graph Data*. John Wiley & Sons.
 - [21] Creswell, A.; Shanahan, M.; and Higgins, I. 2022. Selection-Inference: Exploiting Large Language Models for Interpretable Logical Reasoning. arXiv:2205.09712.

- [22] Dhulipala, L.; Blleloch, G. E.; and Shun, J. 2019. Low-Latency Graph Streaming Using Compressed Purely-Functional Trees. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '19, 918–934.
- [23] Dohan, D.; Xu, W.; Lewkowycz, A.; Austin, J.; Bieber, D.; Lopes, R. G.; Wu, Y.; Michalewski, H.; Saurous, R. A.; Sohl-Dickstein, J.; Murphy, K.; and Sutton, C. 2022. Language Model Cascades. In *Beyond Bayes: Paths Towards Universal Reasoning Systems*, Workshop at ICML '22.
- [24] Drori, I.; Zhang, S.; Shuttlesworth, R.; Tang, L.; Lu, A.; Ke, E.; Liu, K.; Chen, L.; Tran, S.; Cheng, N.; Wang, R.; Singh, N.; Patti, T. L.; Lynch, J.; Shporer, A.; Verma, N.; Wu, E.; and Strang, G. 2022. A neural network solves, explains, and generates university math problems by program synthesis and few-shot learning at human level. *Proceedings of the National Academy of Sciences*, 119(32): e2123433119.
- [25] Fan, W.; Li, J.; Ma, S.; Tang, N.; Wu, Y.; and Wu, Y. 2010. Graph Pattern Matching: From Intractable to Polynomial Time. *Proc. VLDB Endow.*, 3(1–2): 264–275.
- [26] Feng, G.; Meng, X.; and Ammar, K. 2015. DISTINGER: A distributed graph data structure for massive dynamic graph processing. In *Proceedings of the IEEE International Conference on Big Data*, Big Data '15, 1814–1822.
- [27] Friggeri, A.; Chelius, G.; and Fleury, E. 2011. Triangles to Capture Social Cohesion. In *Proceedings of the IEEE Third International Conference on Privacy, Security, Risk and Trust and IEEE Third International Conference on Social Computing*, PASSAT/SocialCom '11, 258–265.
- [28] Friston, K. 2008. Hierarchical Models in the Brain. *PLOS Computational Biology*, 4(11): 1–24.
- [29] Fu, Y.; Peng, H.; Sabharwal, A.; Clark, P.; and Khot, T. 2022. Complexity-Based Prompting for Multi-Step Reasoning. arXiv:2210.00720.
- [30] Gianinazzi, L.; Fries, M.; Dryden, N.; Ben-Nun, T.; Besta, M.; and Hoefer, T. 2021. Learning Combinatorial Node Labeling Algorithms. arXiv:2106.03594.
- [31] Gregor, D.; and Lumsdaine, A. 2005. Lifting Sequential Graph Algorithms for Distributed-Memory Parallel Computation. *SIGPLAN Not.*, 40(10): 423–437.
- [32] Gregor, D.; and Lumsdaine, A. 2005. The Parallel BGL: A generic library for distributed graph computations. *Parallel Object-Oriented Scientific Computing (POOSC)*.
- [33] Hamilton, W. L.; Ying, R.; and Leskovec, J. 2017. Representation Learning on Graphs: Methods and Applications. *Bulletin of the Technical Committee on Data Engineering*, 40(3): 52–74.
- [34] Hartmann, M.; and Sonntag, D. 2022. A survey on improving NLP models with human explanations. In *Proceedings of the First Workshop on Learning with Natural Language Supervision*, 40–47. Association for Computational Linguistics.
- [35] Horváth, T.; Gärtner, T.; and Wrobel, S. 2004. Cyclic Pattern Kernels for Predictive Graph Mining. In *Proceedings of the Tenth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '04, 158–167.
- [36] Huang, W.; Abbeel, P.; Pathak, D.; and Mordatch, I. 2022. Language Models as Zero-Shot Planners: Extracting Actionable Knowledge for Embodied Agents. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, 9118–9147. PMLR.
- [37] Huang, W.; Xia, F.; Xiao, T.; Chan, H.; Liang, J.; Florence, P.; Zeng, A.; Tompson, J.; Mordatch, I.; Chebotar, Y.; Sermanet, P.; Brown, N.; Jackson, T.; Luu, L.; Levine, S.; Hausman, K.; and Ichter, B. 2022. Inner Monologue: Embodied Reasoning through Planning with Language Models. arXiv:2207.05608.
- [38] Jiang, C.; Coenen, F.; and Zito, M. 2013. A survey of frequent subgraph mining algorithms. *The Knowledge Engineering Review*, 28(1): 75–105.
- [39] Kim, G.; Baldi, P.; and McAleer, S. 2023. Language Models can Solve Computer Tasks. arXiv:2303.17491.
- [40] Lertvittayakumjorn, P.; and Toni, F. 2021. Explanation-Based Human Debugging of NLP Models: A Survey. *Transactions of the Association for Computational Linguistics*, 9: 1508–1528.
- [41] Lester, B.; Al-Rfou, R.; and Constant, N. 2021. The Power of Scale for Parameter-Efficient Prompt Tuning. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, EMNLP '21, 3045–3059. Association for Computational Linguistics.
- [42] Li, X. L.; and Liang, P. 2021. Prefix-Tuning: Optimizing Continuous Prompts for Generation. arXiv:2101.00190.
- [43] Long, J. 2023. Large Language Model Guided Tree-of-Thought. arXiv:2305.08291.
- [44] Lumsdaine, A.; Gregor, D.; Hendrickson, B.; and Berry, J. 2007. Challenges in Parallel Graph Processing. *Parallel Processing Letters*, 17(1): 5–20.
- [45] Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegrefe, S.; Alon, U.; Dziri, N.; Prabh-moye, S.; Yang, Y.; Gupta, S.; Majumder, B. P.; Hermann, K.; Welleck, S.; Yazdanbakhsh, A.; and Clark, P. 2023. Self-Refine: Iterative Refinement with Self-Feedback. arXiv:2303.17651.
- [46] Malewicz, G.; Austern, M. H.; Bik, A. J.; Dehnert, J. C.; Horn, I.; Leiser, N.; and Czajkowski, G. 2010.

- Pregel: A System for Large-Scale Graph Processing. In *Proceedings of the International Conference on Management of Data*, SIGMOD '10, 135–146. ACM.
- [47] Ning, X.; Lin, Z.; Zhou, Z.; Wang, Z.; Yang, H.; and Wang, Y. 2023. Skeleton-of-Thought: Large Language Models Can Do Parallel Decoding. arXiv:2307.15337.
- [48] Nye, M.; Andreassen, A. J.; Gur-Ari, G.; Michalewski, H.; Austin, J.; Bieber, D.; Dohan, D.; Lewkowycz, A.; Bosma, M.; Luan, D.; Sutton, C.; and Odena, A. 2021. Show Your Work: Scratchpads for Intermediate Computation with Language Models. arXiv:2112.00114.
- [49] Paul, D.; Ismayilzada, M.; Peyrard, M.; Borges, B.; Bosselut, A.; West, R.; and Faltings, B. 2023. REFINER: Reasoning Feedback on Intermediate Representations. arXiv:2304.01904.
- [50] Prat-Pérez, A.; Dominguez-Sal, D.; Brunat, J. M.; and Larriba-Pey, J.-L. 2012. Shaping Communities out of Triangles. In *Proceedings of the 21st ACM International Conference on Information and Knowledge Management*, CIKM '12, 1677–1681.
- [51] Qiao, S.; Ou, Y.; Zhang, N.; Chen, X.; Yao, Y.; Deng, S.; Tan, C.; Huang, F.; and Chen, H. 2023. Reasoning with Language Model Prompting: A Survey. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, ACL '23, 5368–5393. Association for Computational Linguistics.
- [52] qrdlgit. 2023. graph-of-thoughts Repository. <https://github.com/qrdlgit/graph-of-thoughts>. Accessed: 2023-10-11.
- [53] Radford, A.; Narasimhan, K.; Salimans, T.; and Sutskever, I. 2018. Improving Language Understanding by Generative Pre-Training. <https://openai.com/research/language-unsupervised>. Accessed: 2023-09-06.
- [54] Radford, A.; Wu, J.; Child, R.; Luan, D.; Amodei, D.; and Sutskever, I. 2019. Language Models are Unsupervised Multitask Learners. <https://openai.com/research/better-language-models>. Accessed: 2023-09-06.
- [55] Robinson, I.; Webber, J.; and Eifrem, E. 2015. *Graph Databases: New Opportunities for Connected Data*. O'Reilly Media, 2nd edition.
- [56] Sakr, S.; Bonifati, A.; Voigt, H.; Iosup, A.; Ammar, K.; Angles, R.; Aref, W.; Arenas, M.; Besta, M.; Boncz, P. A.; Daudjee, K.; Valle, E. D.; Dumbra, S.; Hartig, O.; Haslhofer, B.; Hegeman, T.; Hidders, J.; Hose, K.; Iamnitich, A.; Kalavri, V.; Kapp, H.; Martens, W.; Özsu, M. T.; Peukert, E.; Planktikow, S.; Ragab, M.; Ripeanu, M. R.; Salihoglu, S.; Schulz, C.; Selmer, P.; Sequeda, J. F.; Shinavier, J.; Szárnyas, G.; Tommasini, R.; Tumeo, A.; Uta, A.; Varbanescu, A. L.; Wu, H.-Y.; Yakovets, N.; Yan, D.; and Yoneki, E. 2021. The Future is Big Graphs: A Community View on Graph Processing Systems. *Commun. ACM*, 64(9): 62–71.
- [57] Scarselli, F.; Gori, M.; Tsoi, A. C.; Hagenbuchner, M.; and Monfardini, G. 2008. The Graph Neural Network Model. *IEEE Transactions on Neural Networks*, 20(1): 61–80.
- [58] Schaeffer, S. E. 2007. Graph clustering. *Computer Science Review*, 1(1): 27–64.
- [59] Shin, T.; Razeghi, Y.; Logan IV, R. L.; Wallace, E.; and Singh, S. 2020. AutoPrompt: Eliciting Knowledge from Language Models with Automatically Generated Prompts. arXiv:2010.15980.
- [60] Shinn, N.; Labash, B.; and Gopinath, A. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366.
- [61] Shum, K.; Diao, S.; and Zhang, T. 2023. Automatic Prompt Augmentation and Selection with Chain-of-Thought from Labeled Data. arXiv:2302.12822.
- [62] Teixeira, C. H. C.; Fonseca, A. J.; Serafini, M.; Siganos, G.; Zaki, M. J.; and Aboulmaga, A. 2015. Arabesque: A System for Distributed Graph Mining. In *Proceedings of the 25th Symposium on Operating Systems Principles*, SOSP '15, 425–440. ACM.
- [63] Touvron, H.; Lavril, T.; Izacard, G.; Martinet, X.; Lachaux, M.-A.; Lacroix, T.; Rozière, B.; Goyal, N.; Hambro, E.; Azhar, F.; Rodriguez, A.; Joulin, A.; Grave, E.; and Lample, G. 2023. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971.
- [64] Touvron, H.; Martin, L.; Stone, K.; Albert, P.; Almahairi, A.; Babaei, Y.; Bashlykov, N.; Batra, S.; Bhargava, P.; Bhosale, S.; Bikel, D.; Blecher, L.; Ferrer, C. C.; Chen, M.; Cucurull, G.; Esioibu, D.; Fernandes, J.; Fu, J.; Fu, W.; Fuller, B.; Gao, C.; Goswami, V.; Goyal, N.; Hartshorn, A.; Hosseini, S.; Hou, R.; Inan, H.; Kardaş, M.; Kerkez, V.; Khabsa, M.; Kloumann, I.; Korenev, A.; Koura, P. S.; Lachaux, M.-A.; Lavril, T.; Lee, J.; Liskovich, D.; Lu, Y.; Mao, Y.; Martinet, X.; Mihaylov, T.; Mishra, P.; Molybog, I.; Nie, Y.; Poulton, A.; Reizenstein, J.; Rungta, R.; Saladi, K.; Schelten, A.; Silva, R.; Smith, E. M.; Subramanian, R.; Tan, X. E.; Tang, B.; Taylor, R.; Williams, A.; Kuan, J. X.; Xu, P.; Yan, Z.; Zarov, I.; Zhang, Y.; Fan, A.; Kambadur, M.; Narang, S.; Rodriguez, A.; Stojnic, R.; Edunov, S.; and Scialom, T. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288.
- [65] Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, Ł.; and Polosukhin, I. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems (NIPS '17)*, volume 30. Curran Associates.
- [66] Wang, L.; Xu, W.; Lan, Y.; Hu, Z.; Lan, Y.; Lee, R. K.-W.; and Lim, E.-P. 2023. Plan-and-

- Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics, ACL '23*, 2609–2634. Association for Computational Linguistics.
- [67] Wang, X.; Wei, J.; Schuurmans, D.; Le, Q. V.; Chi, E. H.; Narang, S.; Chowdhery, A.; and Zhou, D. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *Proceedings of the Eleventh International Conference on Learning Representations, ICLR '23*.
- [68] Wang, Z.; Cai, S.; Chen, G.; Liu, A.; Ma, X.; and Liang, Y. 2023. Describe, Explain, Plan and Select: Interactive Planning with Large Language Models Enables Open-World Multi-Task Agents. In *Advances in Neural Information Processing Systems (NeurIPS '23)*, volume 36. Curran Associates.
- [69] Wang, Z.; Zhang, G.; Yang, K.; Shi, N.; Zhou, W.; Hao, S.; Xiong, G.; Li, Y.; Sim, M. Y.; Chen, X.; Zhu, Q.; Yang, Z.; Nik, A.; Liu, Q.; Lin, C.; Wang, S.; Liu, R.; Chen, W.; Xu, K.; Liu, D.; Guo, Y.; and Fu, J. 2023. Interactive Natural Language Processing. arXiv:2305.13246.
- [70] Wang, Z. J.; Choi, D.; Xu, S.; and Yang, D. 2021. Putting Humans in the Natural Language Processing Loop: A Survey. In *Proceedings of the First Workshop on Bridging Human-Computer Interaction and Natural Language Processing*, 47–52. Association for Computational Linguistics.
- [71] Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Chi, E.; Le, Q.; and Zhou, D. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv:2201.11903.
- [72] Wu, T.; Jiang, E.; Donsbach, A.; Gray, J.; Molina, A.; Terry, M.; and Cai, C. J. 2022. PromptChainer: Chaining Large Language Model Prompts through Visual Programming. In *Extended Abstracts of the Conference on Human Factors in Computing Systems, CHI EA '22*. ACM.
- [73] Wu, T.; Terry, M.; and Cai, C. J. 2022. AI Chains: Transparent and Controllable Human-AI Interaction by Chaining Large Language Model Prompts. In *Proceedings of the Conference on Human Factors in Computing Systems, CHI '22*. ACM.
- [74] Wu, Z.; Pan, S.; Chen, F.; Long, G.; Zhang, C.; and Yu, P. S. 2021. A Comprehensive Survey on Graph Neural Networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1): 4–24.
- [75] Xie, Y.; Kawaguchi, K.; Zhao, Y.; Zhao, X.; Kan, M.-Y.; He, J.; and Xie, Q. 2023. Self-Evaluation Guided Beam Search for Reasoning. In *Advances in Neural Information Processing Systems (NeurIPS '23)*, volume 36. Curran Associates.
- [76] Yang, S.; Nachum, O.; Du, Y.; Wei, J.; Abbeel, P.; and Schuurmans, D. 2023. Foundation Models for Decision Making: Problems, Methods, and Opportunities. arXiv:2303.04129.
- [77] Yao, S.; Yu, D.; Zhao, J.; Shafran, I.; Griffiths, T. L.; Cao, Y.; and Narasimhan, K. R. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS '23)*, volume 36. Curran Associates.
- [78] Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K. R.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of the Eleventh International Conference on Learning Representations, ICLR '23*.
- [79] Yao, Y.; Li, Z.; and Zhao, H. 2023. Beyond Chain-of-Thought, Effective Graph-of-Thought Reasoning in Large Language Models. arXiv:2305.16582.
- [80] Zelikman, E.; Wu, Y.; Mu, J.; and Goodman, N. 2022. STaR: Bootstrapping Reasoning With Reasoning. In *Advances in Neural Information Processing Systems (NeurIPS '22)*, volume 35, 15476–15488. Curran Associates.
- [81] Zhang, S.; Chen, Z.; Shen, Y.; Ding, M.; Tenenbaum, J. B.; and Gan, C. 2023. Planning with Large Language Models for Code Generation. In *Proceedings of the Eleventh International Conference on Learning Representations, ICLR '23*.
- [82] Zhang, Z.; Cui, P.; and Zhu, W. 2022. Deep Learning on Graphs: A Survey. *IEEE Transactions on Knowledge and Data Engineering*, 34(1): 249–270.
- [83] Zhou, J.; Cui, G.; Hu, S.; Zhang, Z.; Yang, C.; Liu, Z.; Wang, L.; Li, C.; and Sun, M. 2020. Graph neural networks: A review of methods and applications. *AI Open*, 1: 57–81.
- [84] Zhou, Y.; Muresanu, A. I.; Han, Z.; Paster, K.; Pitis, S.; Chan, H.; and Ba, J. 2022. Large Language Models Are Human-Level Prompt Engineers. arXiv:2211.01910.
- [85] Zhu, X.; Wang, J.; Zhang, L.; Zhang, Y.; Huang, Y.; Gan, R.; Zhang, J.; and Yang, Y. 2023. Solving Math Word Problems via Cooperative Reasoning induced Language Models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics, ACL '23*, 4471–4485. Association for Computational Linguistics.

A 正分评分的评估

以下两幅图分别绘制图 5 和图 6 的同一组数据，但采用第 5.1 节和第 5.2 节描述的“正分评分”。

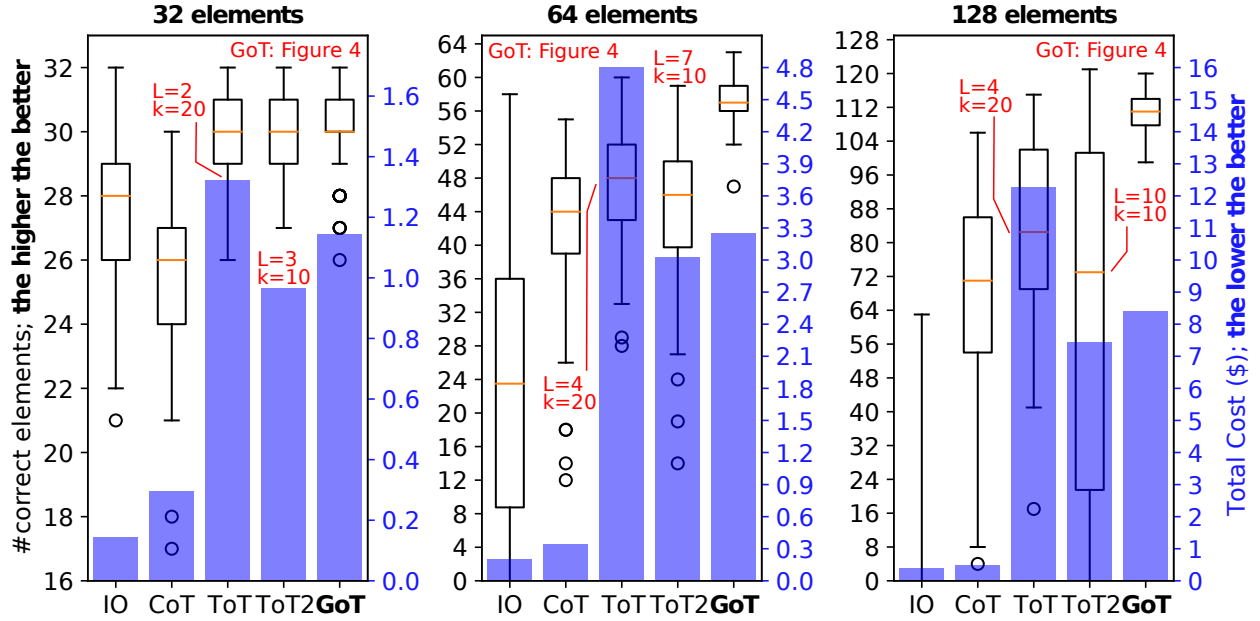


图 9: 使用 ChatGPT-3.5 进行排序任务的准确率和成本。 L 和 k 表示 ToT 的结构 (见第 3.2 节和第 6 节)。图内英文对照见文末译者补充。

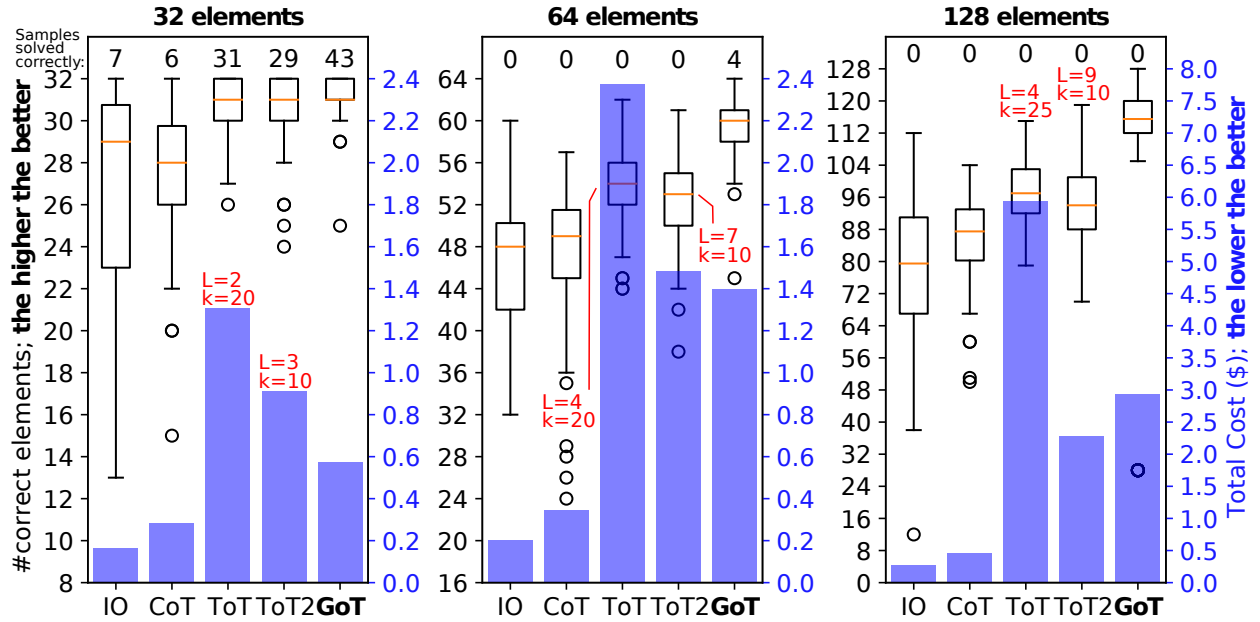


图 10: 使用 ChatGPT-3.5 进行集合求交的准确率和成本。 L 和 k 表示 ToT 的结构 (见第 3.2 节和第 6 节)。图内英文对照见文末译者补充。

B 提示示例——排序

这里只展示 32 元素列表排序的提示，因为 64 元素和 128 元素列表的提示与之相同，唯一的区别是 *split_prompt* 中单样本示例的元素个数与问题规模一致。

在排序任务中，我们使用三种不同的操作与 LLM 交互，每种操作都有相应提示。首先是**生成 (Generate)** 操作，使用 *sort_prompt* 引导 LLM 对给定数值列表排序，并使用 *split_prompt* 引导 LLM 将指定列表拆成指定数量的子列表。接着，**改进 (Improve)** 操作使用 *improve_prompt* 指示 LLM 在发现错误时修正有序列表。最后，**聚合 (Aggregate)** 操作使用 *merge_prompt* 引导 LLM 将两个已经排好序的列表合并为一个有序列表。

首先展示提示模板（表 3），它们用于在运行时动态生成适当的提示。为便于理解，其对应的少样本示例单列于表 4。随后展示求解排序用例过程中与 LLM 的各次交互（表 5 至表 9）。

表 3: 排序任务的提示模板；单层花括号中的参数会在运行时被替换。

sort_prompt: <Instruction> 将以下数字列表按升序排序。只输出排序后的数字列表，不要输出额外文字。
</Instruction>

<Examples> 见表 4 </Examples>

输入: {input_list}

split_prompt (32 个元素): <Instruction> 将以下含 32 个数字的列表拆成两个各含 16 个数字的列表，第一个列表包含前 16 个数字，第二个列表包含后 16 个数字。

只按以下格式输出最终的两个列表，不要输出额外文字或思考！

```
{{  
  "List 1": [3, 4, 3, 5, 7, 8, 1, ...],  
  "List 2": [2, 9, 2, 4, 7, 1, 5, ...]  
}}
```

</Instruction>

<Examples> 见表 4 </Examples>

输入: {input_list}

improve_prompt: <Instruction> 以下两个列表分别表示一个未排序的数字列表及其排序后的版本。排序后的版本有误。请修正该版本，使其正确。确保输出列表按升序排列，元素个数与输入列表相同 ({length})，并且所含元素与输入列表相同。</Instruction>

<Approach>

按以下步骤修正排序错误的列表：

1. 对 0 到 9 的每个数字，比较它在排序错误列表与输入列表中的出现次数。
2. 遍历排序错误的列表，按需添加或删除数字，使每个数字在该列表中的出现次数与输入列表一致。

</Approach>

<Examples> 见表 4 </Examples>

输入: {input_list}

排序错误的列表: {sorted_list}

merge_prompt: <Instruction> 采用归并排序式的方法，将以下两个长度均为 {length} 的有序列表合并为一个长度为 {length_combined} 的有序列表。只输出最终合并后的列表，不要输出额外文字或思考！</Instruction>

<Approach>

按以下步骤以归并排序式的方法合并两个列表：

1. 比较两个列表的第一个元素。
2. 把较小元素追加到合并列表中，并将该元素所在列表的当前位置移到下一个元素。
3. 重复步骤 1 和 2，直到其中一个列表为空。
4. 将非空列表中的剩余元素追加到合并列表中。

</Approach>

将以下两个列表合并为一个有序列表：

1. {input_list1}
2. {input_list2}

合并后的列表：

表 4: 排序任务中各个提示的少样本示例；为简洁起见，部分列表已截短。

sort_prompt:

<Examples>

输入: [5, 1, 0, 1, 2, 0, 4, 8, 1, 9, 5, 1, 3, 3, 9, 7]

输出: [0, 0, 1, 1, 1, 1, 2, 3, 3, 4, 5, 5, 7, 8, 9, 9]

输入: [3, 7, 0, 2, 8, 1, 2, 2, 2, 4, 7, 8, 5, 5, 3, 9, 4, 3, ... (省略 32 个数字中的 14 个)]

输出: [0, 0, 1, 1, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 4, 4, 4, ... (省略 32 个数字中的 14 个)]

输入: [4, 4, 9, 7, 9, 7, 0, 0, 4, 9, 1, 7, 9, 5, 8, 7, 5, 6, ... (省略 64 个数字中的 46 个)]

输出: [0, 0, 0, 0, 0, 0, 0, 1, 1, 2, 2, 2, 2, 2, 3, 3, 3, ... (省略 64 个数字中的 46 个)]

</Examples>

split_prompt (32 个元素):

<Examples>

输入: [9, 6, 7, 7, 2, 0, 2, 2, 3, 5, 0, 9, 2, 2, 4, 4, 5, 2, ... (省略 32 个数字中的 14 个)]

输出:

{{

 "List 1": [9, 6, 7, 7, 2, 0, 2, 2, 3, 5, 0, 9, 2, 2, 4, 4],

 "List 2": [5, 2, 5, 1, 2, 8, 3, 8, 3, 9, 6, 0, 4, 2, 2, 3]

}}

</Examples>

improve_prompt:

<Examples>

输入: [3, 7, 0, 2, 8, 1, 2, 2, 2, 4, 7, 8, 5, 5, 3, 9]

排序错误的列表: [0, 0, 0, 0, 0, 1, 2, 2, 3, 3, 4, 4, 4, 5, 5, 7, 7, 8, 8, 9, 9, 9, 9]

原因: 排序错误的列表多出四个 0、两个 4 和三个 9，并且少了两个 2。

输出: [0, 1, 2, 2, 2, 2, 3, 3, 4, 5, 5, 7, 7, 8, 8, 9]

输入: [6, 4, 5, 7, 5, 6, 9, 7, 6, 9, 4, 6, 9, 8, 1, 9, 2, 4, ... (省略 32 个数字中的 14 个)]

排序错误的列表: [0, 1, 1, 2, 2, 3, 4, 4, 4, 4, 4, 5, 5, 5, 5, 6, 6, 6, ... (省略 32 个数字中的 14 个)]

原因: 排序错误的列表多出两个 4，并且少了两个 6 和一个 9。

输出: [0, 1, 1, 2, 2, 3, 4, 4, 4, 5, 5, 5, 5, 6, 6, 6, 6, 6, ... (省略 32 个数字中的 14 个)]

输入: [4, 4, 9, 7, 9, 7, 0, 0, 4, 9, 1, 7, 9, 5, 8, 7, 5, 6, ... (省略 64 个数字中的 46 个)]

排序错误的列表: [0, 0, 0, 0, 0, 0, 0, 1, 1, 2, 2, 2, 2, 3, 3, 3, 4, 4, ... (省略 64 个数字中的 46 个)]

原因: 排序错误的列表多出一个 8，并且少了两个 2、一个 3、三个 4、两个 5、一个 6、六个 7 和一个 9。

输出: [0, 0, 0, 0, 0, 0, 0, 1, 1, 2, 2, 2, 2, 2, 3, 3, 3, ... (省略 64 个数字中的 46 个)]

</Examples>

merge_prompt: -

表 5: 32 元素列表的排序：执行计划 (GoO)

- GoO:**
- 1. 将输入列表拆成两个大小相同的子列表 (split_prompt)
 - 2. 对每个子列表：对子列表排序 (sort_prompt) 共五次；为每次排序尝试评分；保留最佳结果
 - 3. 将有序子列表合并为一个完整的有序列表 (merge_prompt) 共 10 次；为每次合并尝试评分；保留最佳结果
 - 4. 修正有序列表中可能存在的错误 (improve_prompt) 共 10 次；为每次改进尝试评分；保留最佳结果

表 6: 32 元素列表的排序：步骤 1 (提示/响应)

步骤 1 – 提示:

<Instruction> 将以下含 32 个数字的列表拆成两个各含 16 个数字的列表，第一个列表包含前 16 个数字，第二个列表包含后 16 个数字。

只按以下格式输出最终的两个列表，不要输出额外文字或思考！

```
{{
  "List 1": [3, 4, 3, 5, 7, 8, 1, ...],
  "List 2": [2, 9, 2, 4, 7, 1, 5, ...]
}}
```

</Instruction>

<Examples> 见表 4 </Examples>

输入: [8, 7, 1, 1, 1, 1, 3, 3, 0, 9, 4, 1, 0, 2, 5, 1, 0, 5, 6, 7, 1, 4, 5, 9, 4, 6, 2, 5, 8, 6, 2, 6]

步骤 1 – 响应 (完全正确):

```
{{
  "List 1": [8, 7, 1, 1, 1, 1, 3, 3, 0, 9, 4, 1, 0, 2, 5, 1],
  "List 1": [0, 5, 6, 7, 1, 4, 5, 9, 4, 6, 2, 5, 8, 6, 2, 6]
}}
```

表 7: 32 元素列表的排序: 步骤 2 (提示/响应)

步骤 2a – 提示:

<Instruction> 将以下数字列表按升序排序。只输出排序后的数字列表, 不要输出额外文字。</Instruction>

<Examples> 见表 4 </Examples>

输入: [8, 7, 1, 1, 1, 1, 3, 3, 0, 9, 4, 1, 0, 2, 5, 1]

步骤 2a – 5 个响应:

1. [0, 0, 1, 1, 1, 1, 1, 1, 2, 3, 3, 4, 5, 7, 8, 9] (完全正确)
2. [0, 0, 1, 1, 1, 1, 1, 2, 3, 3, 4, 5, 7, 8, 9] (1 个错误: 少了一个 1)
3. [0, 0, 1, 1, 1, 1, 1, 2, 3, 3, 4, 5, 7, 8, 9] (1 个错误: 少了一个 1)
4. [0, 0, 1, 1, 1, 1, 1, 2, 3, 3, 4, 5, 7, 8, 9] (1 个错误: 少了一个 1)
5. [0, 0, 1, 1, 1, 1, 1, 2, 3, 3, 4, 5, 7, 8, 9] (1 个错误: 少了一个 1)

步骤 2b – 提示:

<Instruction> 将以下数字列表按升序排序。只输出排序后的数字列表, 不要输出额外文字。</Instruction>

<Examples> 见表 4 </Examples>

输入: [0, 5, 6, 7, 1, 4, 5, 9, 4, 6, 2, 5, 8, 6, 2, 6]

步骤 2b – 5 个响应:

1. [0, 1, 2, 2, 4, 4, 5, 5, 5, 6, 6, 6, 7, 8, 9] (1 个错误: 少了一个 6)
 2. [0, 1, 2, 2, 4, 4, 5, 5, 5, 6, 6, 6, 7, 8, 9] (1 个错误: 少了一个 6)
 3. [0, 1, 2, 2, 4, 4, 5, 5, 5, 6, 6, 6, 7, 8, 9] (1 个错误: 少了一个 6)
 4. [0, 1, 2, 2, 4, 4, 5, 5, 5, 6, 6, 6, 6, 7, 8, 9] (完全正确)
 5. [0, 1, 2, 2, 4, 4, 5, 5, 5, 6, 6, 6, 6, 7, 8, 9] (1 个错误: 少了一个 6)
-

表 8: 32 元素列表的排序：步骤 3 (提示/响应)

步骤 3 – 提示:

<Instruction> 采用归并排序式的方法，将以下两个长度均为 16 的有序列表合并为一个长度为 32 的有序列表。只输出最终合并后的列表，不要输出额外文字或思考！ </Instruction>

<Approach>

按以下步骤以归并排序式的方法合并两个列表：

1. 比较两个列表的第一个元素。
2. 把较小元素追加到合并列表中，并将该元素所在列表的当前位置移到下一个元素。
3. 重复步骤 1 和 2，直到其中一个列表为空。
4. 将非空列表中的剩余元素追加到合并列表中。

</Approach>

将以下两个列表合并为一个有序列表：

1. [0, 0, 1, 1, 1, 1, 1, 1, 2, 3, 3, 4, 5, 7, 8, 9]
2. [0, 1, 2, 2, 4, 4, 5, 5, 5, 6, 6, 6, 6, 7, 8, 9]

合并后的列表：

步骤 3 – 10 个响应:

1. [0, 0, 0, 1, 1, 1, 1, 1, 2, 2, 2, 3, 3, 4, 4, 4, 5, 5, 5, 6, 6, 6, 6, 7, 7, 8, 8, 9, 9]
(2 个错误：少了一个 1 和一个 5)
2. [0, 0, 0, 1, 1, 1, 1, 1, 2, 2, 2, 3, 3, 4, 4, 4, 5, 5, 5, 5, 6, 6, 6, 6, 7, 7, 8, 8, 9, 9]
(2 个错误：少了两个 1)
3. [0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 3, 3, 4, 4, 4, 5, 5, 5, 5, 6, 6, 6, 6, 6, 7, 7, 8, 8, 9, 9]
(2 个错误：少了一个 1，并且多出一个 6)
4. [0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 3, 3, 4, 4, 4, 5, 5, 5, 5, 6, 6, 6, 6, 6, 7, 7, 8, 8, 9]
(3 个错误：少了一个 1、一个 9，并且多出一个 6)
5. [0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 3, 3, 4, 4, 4, 5, 5, 5, 5, 6, 6, 6, 6, 6, 7, 7, 8, 8, 9, 9]
(2 个错误：少了一个 1，并且多出一个 6)
6. [0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 3, 3, 4, 4, 4, 5, 5, 5, 6, 6, 6, 6, 6, 7, 7, 8, 8, 9, 9]
(2 个错误：少了一个 1 和一个 5)
7. [0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 3, 3, 4, 4, 4, 5, 5, 5, 5, 6, 6, 6, 6, 6, 7, 7, 8, 8, 9, 9]
(2 个错误：少了一个 1，并且多出一个 6)
8. [0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 3, 3, 4, 4, 4, 5, 5, 5, 5, 6, 6, 6, 6, 6, 7, 7, 8, 8, 9, 9]
(1 个错误：少了一个 1)
9. [0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 3, 3, 4, 4, 4, 5, 5, 5, 5, 6, 6, 6, 6, 6, 7, 7, 8, 8, 9, 9]
(1 个错误：少了一个 1)
10. [0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 3, 3, 4, 4, 4, 5, 5, 5, 5, 6, 6, 6, 6, 6, 7, 7, 8, 8, 9, 9]
(2 个错误：少了一个 1 和一个 5)

表 9: 32 元素列表的排序: 步骤 4 (提示/响应)

步骤 4 – 提示:

<Instruction> 以下两个列表分别表示一个未排序的数字列表及其排序后的版本。排序后的版本有误。请修正该版本, 使其正确。确保输出列表按升序排列, 元素个数与输入列表相同 (32), 并且所含元素与输入列表相同。
</Instruction>

<Approach>

按以下步骤修正排序错误的列表:

1. 对 0 到 9 的每个数字, 比较它在排序错误列表与输入列表中的出现次数。
2. 遍历排序错误的列表, 按需添加或删除数字, 使每个数字在该列表中的出现次数与输入列表一致。

</Approach>

<Examples> 见表 4 </Examples>

输入: [8, 7, 1, 1, 1, 1, 3, 3, 0, 9, 4, 1, 0, 2, 5, 1, 0, 5, 6, 7, 1, 4, 5, 9, 4, 6, 2, 5, 8, 6, 2, 6]

排序错误的列表: [0, 0, 0, 1, 1, 1, 1, 1, 1, 2, 2, 2, 3, 3, 4, 4, 4, 5, 5, 5, 5, 6, 6, 6, 6, 7, 7, 8, 8, 9, 9]

步骤 4 – 10 个响应:

1. 原因: 排序错误的列表少了两个 0、一个 3、一个 4、一个 6 和一个 9; 它还多出一个 1、一个 2 和一个 5。
输出: [0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 3, 3, 4, 5, 5, 5, 6, 6, 6, 6, 7, 7, 8, 8, 9, 9] 4 个错误: 少了一个 1、两个 4 和一个 5
说明: 输出列表的元素个数与输入列表相同, 按升序排列, 并且所含元素与输入列表相同。
2. 输出: [0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 3, 3, 4, 4, 4, 5, 5, 5, 6, 6, 6, 6, 6, 7, 7, 8, 8, 9, 9] 1 个错误: 多出一个 6
3. 输出: [0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 3, 3, 4, 4, 4, 5, 5, 5, 5, 5, 6, 6, 6, 6, 6, 7, 7, 7, 8, 8, 9, 9] 6 个错误: 2、5、6、7、8 和 9 各多出一个
4. 原因: 排序错误的列表少了两个 9 和一个 0。
输出: [0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 3, 3, 4, 4, 4, 5, 5, 5, 5, 6, 6, 6, 6, 7, 7, 8, 8, 9, 9, 9, 9] 3 个错误: 少了一个 1, 并且多出两个 9
5. 原因: 排序错误的列表少了一个 0、一个 3、一个 4、四个 6 和一个 9。
输出: [0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 2, 2, 3, 3, 4, 4, 4, 5, 5, 5, 5, 6, 6, 6, 6, 6, 6, 6, 6, 7, 7, 7, 8, 8, 9, 9, 9] 10 个错误: 1 和 2 各少一个; 多出四个 6、一个 7、一个 8 和两个 9
6. 输出: [0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 3, 3, 3, 4, 4, 4, 5, 5, 5, 5, 6, 6, 6, 6, 6, 7, 7, 7, 7, 8, 8, 8, 9, 9, 9] 8 个错误: 2、3、6 各多出一个, 还多出两个 7、一个 8 和两个 9
7. 输出: [0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 3, 3, 4, 4, 4, 4, 5, 5, 5, 5, 6, 6, 6, 6, 7, 7, 8, 8, 9, 9] 2 个错误: 少了一个 1, 并且多出一个 4
8. 原因: 排序错误的列表少了三个 0、两个 3、两个 4、一个 5、两个 6 和一个 9。
输出: [0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 3, 3, 3, 4, 4, 4, 5, 5, 5, 6, 6, 6, 6, 6, 7, 7, 8, 8, 9] 5 个错误: 1 和 9 各少一个, 多出一个 3 和两个 6
9. 原因: 排序错误的列表少了一个 0、三个 2、两个 3、两个 4、三个 5、两个 6、一个 7 和一个 9。
输出: [0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 3, 3, 3, 4, 4, 4, 5, 5, 5, 5, 6, 6, 6, 6, 7, 8, 8, 9, 9] 8 个错误: 少了一个 1、一个 7; 多出两个 2、两个 3, 以及一个 5 和一个 6
10. 原因: 排序错误的列表少了三个 1、一个 3、一个 4、两个 5、一个 6、一个 7、一个 8 和一个 9。
输出: [0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 3, 3, 4, 4, 4, 5, 5, 5, 5, 6, 6, 6, 7, 7, 8, 8, 9, 9] (完全正确)

最终结果 (排序正确):

[0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 3, 3, 4, 4, 4, 5, 5, 5, 5, 6, 6, 6, 6, 7, 7, 8, 8, 9, 9]

C 提示示例——集合求交

这里只展示两个 32 元素集合求交的提示，因为 64 元素和 128 元素集合的提示与之相同，唯一的区别是 *split_prompt* 的拆分规模按比例调整。

在集合求交任务中，我们使用两种不同的操作与 LLM 交互，每种操作都有相应提示。首先是**生成 (Generate)** 操作，使用 *intersect_prompt* 引导 LLM 对两个输入集合求交，并使用 *split_prompt* 引导 LLM 将指定集合拆成指定数量的不同子集。其次是**聚合 (Aggregate)** 操作，使用 *merge_prompt* 引导 LLM 将两个集合并为一个集合。

首先展示提示模板（表 10），它们用于在运行时动态生成适当的提示。为便于理解，其对应的少样本示例单列于表 11。随后展示一次完整集合求交过程与 LLM 的交互（表 12 至表 15）。

表 10: 集合求交任务的提示模板；单层花括号中的参数会在运行时被替换。

intersect_prompt: <Instruction> 求两个数字集合的交集。只输出两个集合共有的数字组成的集合，不要输出额外文字。</Instruction>

<Examples> 见表 11 </Examples>

输入集合 1: {set1}

输入集合 2: {set2}

split_prompt (32 个元素): <Instruction> 将以下含 32 个数字的列表拆成两个各含 16 个数字的列表，第一个列表包含前 16 个数字，第二个列表包含后 16 个数字。

只按以下格式输出这两个列表，不要输出额外文字或思考！

```
{{  
  "List 1": [13, 16, 30, 6, 21, 7, 31, ...],  
  "List 2": [25, 24, 10, 4, 27, 0, 14, ...]  
}}
```

</Instruction>

<Examples> 见表 11 </Examples>

输入: {input}

merge_prompt: <Instruction> 把第二个列表追加到第一个列表末尾，将以下两个列表合并为一个列表。

只输出最终列表，不要输出额外文字或思考！ </Instruction>

列表 1: {input1}

列表 2: {input2}

表 11: 集合求交任务中各个提示的少样本示例；为简洁起见，部分列表已截短。

intersect_prompt:

<Examples>

输入集合 1: [13, 16, 30, 6, 21, 7, 31, 15, 11, 1, 24, 10, 9, 3, 20, 8]

输入集合 2: [25, 24, 10, 4, 27, 0, 14, 12, 8, 2, 29, 20, 17, 19, 26, 23]

输出: [24, 10, 20, 8]

输入集合 1: [26, 40, 42, 57, 15, 31, 5, 32, 11, 4, 24, 28, 51, 54, ... (省略 32 个数字中的 18 个)]

输入集合 2: [16, 60, 36, 48, 0, 15, 5, 19, 46, 24, 1, 6, 61, 10, ... (省略 32 个数字中的 18 个)]

输出: [40, 15, 5, 24, 35, 59, 16, 63]

输入集合 1: [115, 61, 35, 103, 90, 117, 86, 44, 63, 45, 40, 30, 74, 33, ... (省略 64 个数字中的 50 个)]

输入集合 2: [13, 35, 20, 96, 34, 18, 47, 127, 126, 9, 21, 16, 77, 22, ... (省略 64 个数字中的 50 个)]

输出: [115, 35, 90, 117, 63, 40, 30, 33, 15, 89, 50, 12, 2, 84, 7, 57, 96]

</Examples>

split_prompt (32 个元素):

<Examples>

输入: [26, 40, 42, 57, 15, 31, 5, 32, 11, 4, 24, 28, 51, 54, ... (省略 32 个数字中的 18 个)]

输出:

{{

"List 1": [26, 40, 42, 57, 15, 31, 5, 32, 11, 4, 24, 28, 51, 54, 12, 22],

"List 2": [33, 35, 7, 13, 2, 59, 8, 23, 43, 16, 29, 55, 25, 63, 21, 18]

}}

</Examples>

merge_prompt: -

表 12: 两个 32 元素集合的交集: 执行计划 (GoO)

GoO:

1. 将第二个输入集合拆成两个大小相同的子集 (split_prompt)
 2. 对每个子集: 计算它与第一个输入集合的交集 (intersect_prompt) 共五次; 为每次排序尝试评分; 保留最佳结果
 3. 将得到的各个交集合并为完整的交集 (merge_prompt) 共 10 次; 为每次合并尝试评分; 保留最佳结果
-

表 13: 两个 32 元素集合的交集: 步骤 1 (提示/响应)

步骤 1 – 提示:

<Instruction> 将以下含 32 个数字的列表拆成两个各含 16 个数字的列表, 第一个列表包含前 16 个数字, 第二个列表包含后 16 个数字。

只按以下格式输出这两个列表, 不要输出额外文字或思考!

```
{{  
  "List 1": [13, 16, 30, 6, 21, 7, 31, ...],  
  "List 2": [25, 24, 10, 4, 27, 0, 14, ...]  
}}
```

</Instruction>

<Examples> 见表 11 </Examples>

输入: [31, 11, 4, 63, 38, 58, 59, 24, 61, 14, 32, 39, 27, 46, 48, 19, 52, 57, 50, 56, 3, 2, 53, 29, 5, 37, 62, 41, 36, 12, 49, 16]

步骤 1 – 响应 (完全正确):

```
{{  
  "List 1": [31, 11, 4, 63, 38, 58, 59, 24, 61, 14, 32, 39, 27, 46, 48, 19],  
  "List 1": [52, 57, 50, 56, 3, 2, 53, 29, 5, 37, 62, 41, 36, 12, 49, 16]  
}}
```

表 14: 两个 32 元素集合的交集：步骤 2 (提示/响应)

步骤 2a – 提示:

<Instruction> 求两个数字集合的交集。只输出两个集合共有的数字组成的集合，不要输出额外文字。

</Instruction>

<Examples> 见表 11 </Examples>

输入集合 1: [11, 60, 1, 49, 21, 33, 14, 56, 54, 15, 23, 40, 45, 22, 7, 28, 20, 46, 51, 6, 34, 37, 3, 50, 17, 8, 25, 0, 35, 47, 18, 19]

输入集合 2: [31, 11, 4, 63, 38, 58, 59, 24, 61, 14, 32, 39, 27, 46, 48, 19]

步骤 2a – 5 个响应:

1. [11, 14, 46, 14, 19] (1 个错误: 14 重复出现)
2. **输出:** [11, 14, 46, 19] (完全正确)
3. [11, 14, 46, 14, 19] (1 个错误: 14 重复出现)
4. [11, 14, 46, 19] (完全正确)
5. [11, 14, 46, 14, 19] (1 个错误: 14 重复出现)

步骤 2b – 提示:

<Instruction> 求两个数字集合的交集。只输出两个集合共有的数字组成的集合，不要输出额外文字。

</Instruction>

<Examples> 见表 11 </Examples>

输入集合 1: [11, 60, 1, 49, 21, 33, 14, 56, 54, 15, 23, 40, 45, 22, 7, 28, 20, 46, 51, 6, 34, 37, 3, 50, 17, 8, 25, 0, 35, 47, 18, 19]

输入集合 2: [52, 57, 50, 56, 3, 2, 53, 29, 5, 37, 62, 41, 36, 12, 49, 16]

步骤 2b – 5 个响应:

1. [50, 56, 49] (2 个错误: 少了 3 和 37)
 2. [50, 56, 37, 49] (1 个错误: 少了 3)
 3. [56, 49, 37, 3, 50] (完全正确)
 4. [50, 56, 37, 49] (1 个错误: 少了 3)
 5. [50, 56, 37, 49] (1 个错误: 少了 3)
-

表 15: 两个 32 元素集合的交集: 步骤 3 (提示/响应)

步骤 3 – 提示:

<Instruction> 把第二个列表追加到第一个列表末尾, 将以下两个列表合并为一个列表。

只输出最终列表, 不要输出额外文字或思考! </Instruction>

列表 1: [11, 14, 46, 19]

列表 2: [56, 49, 37, 3, 50]

步骤 3 – 10 个响应:

1. [11, 14, 46, 19, 56, 49, 37, 3, 50] (完全正确)
2. [11, 14, 46, 19, 56, 49, 37, 3, 50] (完全正确)
3. [11, 14, 46, 19, 56, 49, 37, 3, 50] (完全正确)
4. [11, 14, 46, 19, 56, 49, 37, 3, 50] (完全正确)
5. [11, 14, 46, 19, 56, 49, 37, 3, 50] (完全正确)
6. [11, 14, 46, 19, 56, 49, 37, 3, 50] (完全正确)
7. [11, 14, 46, 19, 56, 49, 37, 3, 50] (完全正确)
8. [11, 14, 46, 19, 56, 49, 37, 3, 50] (完全正确)
9. [11, 14, 46, 19, 56, 49, 37, 3, 50] (完全正确)
10. [11, 14, 46, 19, 56, 49, 37, 3, 50] (完全正确)

最终结果 (交集正确):

[11, 14, 46, 19, 56, 49, 37, 3, 50]

D 提示示例——关键词计数

这里只展示关键词计数任务的 GoT4 提示,因为 GoT8 和 GoTx 使用的提示与之相同,唯一的细微区别是 *split_prompt* 中的拆分规模会作调整。

在关键词计数任务中,我们使用三种不同的操作与 LLM 交互,每种操作都有相应提示。首先是**生成 (Generate)**操作,使用 *count_prompt* 引导 LLM 统计文本中的关键词,并使用 *split_prompt* 引导 LLM 将给定文本拆成若干片段。接着,**聚合 (Aggregate)**操作使用 *merge_prompt* 引导 LLM 将两个关键词计数字典合并为一个字典。最后,**验证并改进 (ValidateAndImprove)**操作使用 *improve_merge_prompt* 指示 LLM 修正先前聚合操作产生的错误。

这里展示提示模板(表 16 至表 17),它们用于在运行时动态生成适当的提示。为便于理解,其对应的少样本示例单列于表 18 和表 19。随后展示一次完整关键词计数过程与 LLM 的交互(表 20 至表 28)。

表 16: 关键词计数任务的提示模板;单层花括号中的参数会在运行时被替换。

count_prompt: <Instruction> 统计输入文本中每个国家被明确提及的次数。你可以生成任意中间列表和状态,但最终输出只能包含至少出现一次的各个国家及其频次,采用以下 JSON 格式,并加上前缀”Output: ”(确保输出国家名的拼写与输入文本一致):

```
{{
  "country1": frequency1,
  "country2": frequency2,
  ...
}}
```

</Instruction>

<Approach>

按以下步骤统计每个国家的频次:

1. 将输入文本拆成长度相近的四个段落。
2. 统计每个段落中各国的频次。
3. 将各个段落中同一个国家的频次相加,得到合计频次。

</Approach>

<Examples> 见表 18 </Examples>

输入: {input_text}

split_prompt: <Instruction> 将以下输入文本拆成长度大致相同的 4 个段落。

只按以下格式输出最终的 4 个段落,不要输出额外文字或思考:

```
{{
  "Paragraph 1": " 某个段落的文本 ...",
  "Paragraph 2": " 某个段落的文本 ...",
  "Paragraph 3": " 某个段落的文本 ...",
  "Paragraph 4": " 某个段落的文本 ..."
```

```
}}
```

</Instruction>

<Example> 见表 19 </Example>

输入: {input_text}

表 17: 关键词计数任务的提示模板（续）；单层花括号中的参数会在运行时被替换。

merge_prompt: <Instruction> 将以下两个字典合并成一个字典，每个字典都包含一段文本中各国的出现频次。只需将每个国家的频次相加；如果某个国家未出现在其中一个字典中，则将它加入最终字典，并使用另一个字典中的频次。

只输出最终合并后的字典，不要输出额外文字或思考！ </Instruction>

<Approach>

按以下步骤将两个字典合并成一个字典：

1. 创建一个新字典，用于存储合并后的频次。
2. 遍历第一个字典的键，将每个国家及其频次加入新字典。
3. 遍历第二个字典的键，将每个国家及其频次加入新字典；如果该国家已经存在，就将其频次加到现有值上。

</Approach>

将以下两个字典合并成一个字典：

{dictionary_1}

{dictionary_2}

合并后的输出：

improve_merge_prompt: <Instruction> 以下两个字典被合并成了下方的第三个字典，但合并时出现错误，第三个字典不正确。请修正第三个字典，使其包含各个国家的正确频次。正确频次为前两个字典中该国家频次之和。如果某个国家未出现在其中一个字典中，则将它加入最终字典，并使用另一个字典中的频次。

</Instruction>

<Example> 见表 19 </Example>

字典 1: {dictionary_1}

字典 2: {dictionary_2}

合并错误的字典: {dictionary_incorrect}

输出：

表 18: 关键词计数任务中 **count** 提示的少样本示例；为简洁起见，部分段落和字典已截短，格式略作调整。

count_prompt:

<Examples>

输入：Alexandra 从 Canada 出发，登上了宏大旅程的第一趟航班。她的环球旅行 ... （省略）

段落：

Alexandra 从 Canada 出发，登上了宏大旅程的第一趟航班。她的环球旅行行程 ... （省略）

她的第一站是 Mexico，在那里她惊叹于玛雅遗址。之后，她探索了雨林 ... （省略）

子列表中的频次：

{{ "Canada": 1 }}

{{ "Mexico": 1, "Brazil": 1, "Argentina": 1 }}

输出： {{ "Canada": 1, "Mexico": 1, "Brazil": 1, "Argentina": 1 }}

输入：这段冒险把他带到 Peru 的山峰，他徒步前行，探寻马丘比丘的奥秘 ... （省略）

段落：

这段冒险把他带到 Peru 的山峰，他徒步前行，探寻马丘比丘的奥秘。随后，他 ... （省略）

他短暂绕道 Uruguay 和 Paraguay，体验了当地文化的活力，随后 ... （省略）

子列表：

{{ "Peru": 1, "Chile": 1 }}

{{ "Uruguay": 1, "Paraguay": 1, "Canada": 1, "Peru": 1, "Brazil": 1, "Mexico": 1 }}

输出： {{ "Peru": 2, "Chile": 1, "Uruguay": 1, "Paraguay": 1, "Canada": 1, "Brazil": 1, "Mexico": 1 }}

输入：向西旅行时，她欣赏了 Italy 的艺术，并在 France 品尝咖啡。音乐来自 ... （省略）

段落：

向西旅行时，她欣赏了 Italy 的艺术，并在 France 品尝咖啡。

Spain 的音乐和 Greece 的历史使她更加喜爱欧洲。Norway 的北欧美景，... （省略）

她在 Ireland 跳舞，在 Scotland 探索城堡，并赞叹 Germany 和 Russia 的建筑。

Italy、Norway、Sweden 和 Germany 永远都是她最喜爱的旅行目的地。

子列表：

{{ "Italy": 1, "France": 1 }}

{{ "Spain": 1, "Greece": 1, "Norway": 1, "Sweden": 1, "Finland": 1, "Denmark": 1 }}

{{ "Ireland": 1, "Scotland": 1, "Germany": 1, "Russia": 1 }}

{{ "Italy": 1, "Norway": 1, "Sweden": 1, "Germany": 1 }}

输出： {{ "Italy": 2, "France": 1, "Spain": 1, "Greece": 1, "Norway": 2, "Sweden": 2, ... （省略） }}

</Examples>

表 19: 关键词计数任务中 `split`、`merge` 和 `improve_merge` 提示的少样本示例；为简洁起见，部分段落和字典已截短，格式略作调整。

split_prompt: <Examples> 输入：向西旅行时，她欣赏了 Italy 的艺术，并在 France 品尝咖啡。Spain 的音乐和 Greece 的历史使她更加喜爱欧洲。Norway、Sweden、Finland 和 Denmark 的北欧美景令她惊叹。她在 Ireland 跳舞，在 Scotland 探索城堡，并赞叹 Germany 和 Russia 的建筑。Italy、Norway、Sweden 和 Germany 永远都是她最喜爱的旅行目的地。 输出： <pre>{{ "Paragraph 1": " 向西旅行时，她欣赏了 Italy 的艺术，并在 France 品尝咖啡。", "Paragraph 2": "Spain 的音乐和 Greece 的历史使她更加喜爱 ... （省略）", "Paragraph 3": " 她在 Ireland 跳舞，在 Scotland 探索城堡，并惊叹 ... （省略）", "Paragraph 4": "Italy、Norway、Sweden 和 Germany 永远都是她最喜爱的 ... （省略）" }}</pre> </Examples>
merge_prompt: - improve_merge_prompt: <Example> 字典 1: {{ "Peru": 2, "Chile": 1, "Uruguay": 1, "Paraguay": 1 }} 字典 2: {{ "Peru": 1, "Argentina": 1, "Canada": 1, "Chile": 3, "Germany": 2 }} 合并错误的字典： <pre>{{ "Peru": 3, "Chile": 2, "Uruguay": 1, "Paraguay": 1, "Argentina": 1, "Chile": 3, "Germany": 2 }}</pre> 输出： <pre>{{ "Peru": 3, "Chile": 4, "Uruguay": 1, "Paraguay": 1, "Argentina": 1, "Canada": 1, "Germany": 2 }}</pre> </Example>

表 20: 将文本拆成 4 段的关键词计数示例 (GoT4)：执行计划 (GoO)

GoO: 1. 将输入文本拆成大小大致相同的四个段落 (<code>split_prompt</code>) 2. 对每个段落：统计各个国家的出现次数 (<code>count_prompt</code>) 共 10 次；为每次计数尝试评分；保留最佳结果 3. 将各国计数合并为一个字典 (<code>merge_prompt</code>) 共 3 次；验证并改进无效的合并尝试 (<code>improve_merge_prompt</code>) 每次最多尝试改进 3 次；评分；保留最佳结果

表 21: 将文本拆成 4 段的关键词计数示例 (GoT4): 步骤 1 (提示/响应)

步骤 1 – 提示:

<Instruction> 将以下输入文本拆成长度大致相同的 4 个段落。

只按以下格式输出最终的 4 个段落，不要输出额外文字或思考：

```
{{
  "Paragraph 1": " 某个段落的文本 ... ",
  "Paragraph 2": " 某个段落的文本 ... ",
  "Paragraph 3": " 某个段落的文本 ... ",
  "Paragraph 4": " 某个段落的文本 ... "
}}
```

</Instruction>

<Example> 见表 19 </Example>

输入：我的朋友、来自 Peru 的 Alex 曾讲述他去 Argentina 旅行的经历。在那里，他了解到 Argentina 与 Brazil 因共同的历史而建立了密切的文化联系。他深情地谈起自己在 Argentina 和 Brazil 度过的时光，感叹两国相似的音乐、舞蹈和烹饪传统无缝地跨越了国界。这让我想起一部曾看过的关于横跨 Iran 和 China 的古代丝绸之路的纪录片。它讲到 Iran 与 China 共同的历史叙事，强调几个世纪以来，商品、思想和文化如何在两国间流动。有趣的是，纪录片还指出，这条东方路线与文艺复兴时期 Italy 和 France 之间活跃的交流存在某些相似之处。Italy 与 France 在文化和思想上的纽带确实深厚，正如存在于 United States 与 Canada 之间的坚韧贸易关系。United States 和 Canada 除了地理上相邻，还存在经济纽带，这种纽带强烈影响了两国的政策和国际地位。同样，Australia 与 New Zealand 的联系体现了它们在太平洋地区的整体身份。两国虽然各有特点，但由于文化、语言和殖民历史背景极为相似，Australia 和 New Zealand 往往被视作一个整体。这些相互联系的记述启发我进一步探究历史，随后我发现了一段引人入胜的叙述，讲述 Ukraine 和 Poland 如何在多年共存中相互影响。尽管过去动荡不安，Ukraine 和 Poland 共同拥有一幅深深深入双方历史的文化图景。这与 North Korea 和 South Korea 之间的复杂关系颇为相似：尽管两国存在政治差异，却有共同的遗产和传统。因此，从 Argentina 到 South Korea，世界就像一张复杂的国家网络，各国通过共同的历史、文化，有时还有共同的命运相互交织。

步骤 1 – 响应（完全正确）：

```
{{
  "Paragraph 1": " 我的朋友、来自 Peru 的 Alex ... （省略） ... 跨越了这些国家的边界。",
  "Paragraph 2": " 这让我想起一部 ... （省略） ... United States 与 Canada 之间。",
  "Paragraph 3": "United States 和 Canada 除了 ... （省略） ... 在多年共存之中。",
  "Paragraph 4": " 尽管过去动荡不安 ... （省略） ... 有时还有共同的命运。"
}}
```

表 22: 将文本拆成 4 段的关键词计数示例 (GoT4): 步骤 2a (提示/响应)

步骤 2a – 提示:

<Instruction> 统计输入文本中每个国家被明确提及的次数。你可以生成任意中间列表和状态，但最终输出只能包含至少出现一次的各个国家及其频次，采用以下 JSON 格式，并加上前缀"Output: "（确保输出中国家名的拼写与输入文本一致）:

```
{{
  "country1": frequency1,
  "country2": frequency2,
  ...
}}
```

</Instruction>

<Approach>

按以下步骤统计每个国家的频次:

1. 将输入文本拆成长度相近的四个段落。
2. 统计每个段落中各国的频次。
3. 将各个段落中同一个国家的频次相加，得到合计频次。

</Approach>

<Examples> 见表 18 </Examples>

输入: 我的朋友、来自 Peru 的 Alex 曾讲述他去 Argentina 旅行的经历。在那里，他了解到 Argentina 与 Brazil 因共同的历史而建立了密切的文化联系。他深情地谈起自己在 Argentina 和 Brazil 度过的时光，感叹两国相似的音乐、舞蹈和烹饪传统无缝地跨越了国界。

步骤 2a – 10 个响应:

1. {{ "Peru": 1, "Argentina": 1, "Brazil": 1 }} (3 个错误: 少计两次"Argentina" 和一次"Brazil")
2. {{ "Peru": 1, "Argentina": 2, "Brazil": 2 }} (1 个错误: 少计一次"Argentina")
3. {{ "Peru": 1, "Argentina": 2, "Brazil": 2 }} (1 个错误: 少计一次"Argentina")
4. {{ "Peru": 1, "Argentina": 1, "Brazil": 1 }} (3 个错误: 少计两次"Argentina" 和一次"Brazil")
5. {{ "Peru": 1, "Argentina": 2, "Brazil": 2 }} (1 个错误: 少计一次"Argentina")
6. {{ "Peru": 1, "Argentina": 2, "Brazil": 2 }} (1 个错误: 少计一次"Argentina")
7. {{ "Peru": 1, "Argentina": 1, "Brazil": 1 }} (3 个错误: 少计两次"Argentina" 和一次"Brazil")
8. {{ "Peru": 1, "Argentina": 1, "Brazil": 1 }} (3 个错误: 少计两次"Argentina" 和一次"Brazil")
9. {{ "Peru": 1, "Argentina": 1, "Brazil": 1 }} (3 个错误: 少计两次"Argentina" 和一次"Brazil")
10. {{ "Peru": 1, "Argentina": 1, "Brazil": 1 }} (3 个错误: 少计两次"Argentina" 和一次"Brazil")

表 23: 将文本拆成 4 段的关键词计数示例 (GoT4): **步骤 2b (提示/响应)**

步骤 2b – 提示:

<Instruction> 统计输入文本中每个国家被明确提及的次数。你可以生成任意中间列表和状态, 但最终输出只能包含至少出现一次的各个国家及其频次, 采用以下 JSON 格式, 并加上前缀"Output: " (确保输出中国国家名的拼写与输入文本一致):

```
{{
  "country1": frequency1,
  "country2": frequency2,
  ...
}}
```

</Instruction>

<Approach>

按以下步骤统计每个国家的频次:

1. 将输入文本拆成长度相近的四个段落。
2. 统计每个段落中各国的频次。
3. 将各个段落中同一个国家的频次相加, 得到合计频次。

</Approach>

<Examples> 见表 18 </Examples>

输入: 这让我想起曾看过的一部关于横跨 Iran 和 China 的古代丝绸之路的纪录片。它讲到 Iran 与 China 共同的历史叙事, 强调几个世纪以来, 商品、思想和文化如何在两国间流动。有趣的是, 纪录片还指出, 这条东方路线与文艺复兴时期 Italy 和 France 之间活跃的交流存在某些相似之处。Italy 与 France 在文化和思想上的纽带确实深厚, 正如 United States 与 Canada 之间坚韧的贸易关系。

步骤 2b – 10 个响应:

1. {{ "Iran": 1, "China": 1, "Italy": 1, "France": 1, "United States": 1, "Canada": 1 }}
(4 个错误: "Iran"、"China"、"Italy" 和 "France" 各少计一次)
2. {{ "Iran": 1, "China": 1, "Italy": 1, "France": 1, "United States": 1, "Canada": 1 }}
(4 个错误: "Iran"、"China"、"Italy" 和 "France" 各少计一次)
3. {{ "Iran": 3, "China": 2, "Italy": 1, "France": 1, "United States": 1, "Canada": 1 }}
(3 个错误: "Iran" 多计一次, "Italy" 和 "France" 各少计一次)
4. {{ "Iran": 1, "China": 1, "Italy": 1, "France": 1, "United States": 1, "Canada": 1 }}
(4 个错误: "Iran"、"China"、"Italy" 和 "France" 各少计一次)
5. {{ "Iran": 2, "China": 2, "Italy": 1, "France": 1, "United States": 1, "Canada": 1 }}
(2 个错误: "Italy" 和 "France" 各少计一次)
6. {{ "Iran": 1, "China": 1, "Italy": 1, "France": 1, "United States": 1, "Canada": 1 }}
(4 个错误: "Iran"、"China"、"Italy" 和 "France" 各少计一次)
7. {{ "Iran": 1, "China": 1, "Italy": 1, "France": 1, "United States": 1, "Canada": 1 }}
(4 个错误: "Iran"、"China"、"Italy" 和 "France" 各少计一次)
8. {{ "Iran": 1, "China": 1, "Italy": 1, "France": 1, "United States": 1, "Canada": 1 }}
(4 个错误: "Iran"、"China"、"Italy" 和 "France" 各少计一次)
9. {{ "Iran": 1, "China": 1, "Italy": 1, "France": 1, "United States": 1, "Canada": 1 }}
(4 个错误: "Iran"、"China"、"Italy" 和 "France" 各少计一次)
10. {{ "Iran": 2, "China": 2, "Italy": 1, "France": 2, "United States": 1, "Canada": 1 }}
(1 个错误: "Italy" 少计一次)

表 24: 将文本拆成 4 段的关键词计数示例 (GoT4): **步骤 2c (提示/响应)**

步骤 2c – 提示:

<Instruction> 统计输入文本中每个国家被明确提及的次数。你可以生成任意中间列表和状态, 但最终输出只能包含至少出现一次的各个国家及其频次, 采用以下 JSON 格式, 并加上前缀"Output: " (确保输出中国家名的拼写与输入文本一致):

```
{{
  "country1": frequency1,
  "country2": frequency2,
  ...
}}
```

</Instruction>

<Approach>

按以下步骤统计每个国家的频次:

1. 将输入文本拆成长度相近的四个段落。
2. 统计每个段落中各国的频次。
3. 将各个段落中同一个国家的频次相加, 得到合计频次。

</Approach>

<Examples> 见表 18 </Examples>

输入: United States 和 Canada 除了地理上相邻, 还存在经济纽带, 这种纽带强烈影响了两国的政策和国际地位。同样, Australia 与 New Zealand 的联系体现了它们在太平洋地区的整体身份。两国虽然各有特点, 但由于文化、语言和殖民历史背景极为相似, Australia 和 New Zealand 往往被视作一个整体。这些相互联系的记述启发我进一步探究历史, 随后我发现了一段引人入胜的叙述, 讲述 Ukraine 和 Poland 如何在多年共存中相互影响。

步骤 2c – 10 个响应:

1. {{ "United States": 1, "Canada": 1, "Australia": 2, "New Zealand": 2, "Ukraine": 1, "Poland": 1 }}
(完全正确)
2. {{ "United States": 1, "Canada": 1, "Australia": 2, "New Zealand": 2, "Ukraine": 1, "Poland": 1 }}
(完全正确)
3. {{ "United States": 1, "Canada": 1, "Australia": 1, "New Zealand": 1, "Australia and New Zealand": 1, "Ukraine": 1, "Poland": 1 }}
(3 个错误: "Australia"、"New Zealand" 各少计一次, "Australia and New Zealand" 多计一次)
4. {{ "United States": 1, "Canada": 1, "Australia": 2, "New Zealand": 2, "Ukraine": 1, "Poland": 1 }}
(完全正确)
5. {{ "United States": 1, "Canada": 1, "Australia": 1, "New Zealand": 1, "Australia and New Zealand": 1, "Ukraine": 1, "Poland": 1 }}
(3 个错误: "Australia"、"New Zealand" 各少计一次, "Australia and New Zealand" 多计一次)
6. {{ "United States": 1, "Canada": 1, "Australia": 1, "New Zealand": 1, "Ukraine": 1, "Poland": 1 }}
(2 个错误: "Australia" 和 "New Zealand" 各少计一次)
7. {{ "United States": 1, "Canada": 1, "Australia": 2, "New Zealand": 2, "Ukraine": 1, "Poland": 1 }}
(完全正确)
8. {{ "United States": 1, "Canada": 1, "Australia": 1, "New Zealand": 1, "Ukraine": 1, "Poland": 1 }}
(2 个错误: "Australia" 和 "New Zealand" 各少计一次)
9. {{ "United States": 1, "Canada": 1, "Australia": 2, "New Zealand": 2, "Ukraine": 1, "Poland": 1 }}
(完全正确)
10. {{ "United States": 1, "Canada": 1, "Australia": 2, "New Zealand": 2, "Ukraine": 1, "Poland": 1 }}
(完全正确)

表 25: 将文本拆成 4 段的关键词计数示例 (GoT4): 步骤 2d (提示/响应)

步骤 2d – 提示:

<Instruction> 统计输入文本中每个国家被明确提及的次数。你可以生成任意中间列表和状态, 但最终输出只能包含至少出现一次的各个国家及其频次, 采用以下 JSON 格式, 并加上前缀"Output: " (确保输出中国国家名的拼写与输入文本一致):

```
{{
  "country1": frequency1,
  "country2": frequency2,
  ...
}}
```

</Instruction>

<Approach>

按以下步骤统计每个国家的频次:

1. 将输入文本拆成长度相近的四个段落。
2. 统计每个段落中各国的频次。
3. 将各个段落中同一个国家的频次相加, 得到合计频次。

</Approach>

<Examples> 见表 18 </Examples>

输入: 尽管过去动荡不安, Ukraine 和 Poland 共同拥有一幅深深织入双方历史的文化图景。这与 North Korea 和 South Korea 之间的复杂关系颇为相似: 尽管两国存在政治差异, 却有共同的遗产和传统。因此, 从 Argentina 到 South Korea, 世界就像一张复杂的国家网络, 各国通过共同的历史、文化, 有时还有共同的命运相互交织。

步骤 2d – 10 个响应:

1. {{ "Ukraine": 1, "Poland": 1, "North Korea": 1, "South Korea": 2, "Argentina": 1 }} (完全正确)
2. {{ "Ukraine": 1, "Poland": 1, "North Korea": 1, "South Korea": 2, "Argentina": 1 }} (完全正确)
3. {{ "Ukraine": 1, "Poland": 1, "North Korea": 1, "South Korea": 2, "Argentina": 1 }} (完全正确)
4. {{ "Ukraine": 1, "Poland": 1, "North Korea": 1, "South Korea": 2, "Argentina": 1 }} (完全正确)
5. {{ "Ukraine": 1, "Poland": 1, "North Korea": 1, "South Korea": 2, "Argentina": 1 }} (完全正确)
6. {{ "Ukraine": 1, "Poland": 1, "North Korea": 1, "South Korea": 2, "Argentina": 1 }} (完全正确)
7. {{ "Ukraine": 1, "Poland": 1, "North Korea": 1, "South Korea": 2, "Argentina": 1 }} (完全正确)
8. {{ "Ukraine": 1, "Poland": 1, "North Korea": 1, "South Korea": 2, "Argentina": 1 }} (完全正确)
9. {{ "Ukraine": 1, "Poland": 1, "North Korea": 1, "South Korea": 2, "Argentina": 1 }} (完全正确)
10. {{ "Ukraine": 1, "Poland": 1, "North Korea": 1, "South Korea": 2, "Argentina": 1 }} (完全正确)

表 26: 将文本拆成 4 段的关键词计数示例 (GoT4): 步骤 3a (提示/响应)

步骤 3a – 提示:

<Instruction> 将以下两个字典合并成一个字典，每个字典都包含一段文本中各国的出现频次。只需将每个国家的频次相加；如果某个国家未出现在其中一个字典中，则将它加入最终字典，并使用另一个字典中的频次。

只输出最终合并后的字典，不要输出额外文字或思考！ </Instruction>

<Approach>

按以下步骤将两个字典合并成一个字典：

1. 创建一个新字典，用于存储合并后的频次。
2. 遍历第一个字典的键，将每个国家及其频次加入新字典。
3. 遍历第二个字典的键，将每个国家及其频次加入新字典；如果该国已经存在，就将其频次加到现有值上。

</Approach>

将以下两个字典合并成一个字典：

{ { "Peru": 1, "Argentina": 2, "Brazil": 2 } }

{ { "Iran": 2, "China": 2, "Italy": 1, "France": 2, "United States": 1, "Canada": 1 } }

合并后的输出：

步骤 3a – 3 个响应:

1. { { "Peru": 1, "Argentina": 2, "Brazil": 2, "Iran": 2, "China": 2, "Italy": 1, "France": 2, "United States": 1, "Canada": 1 } } (有效, 无需改进) (2 个错误, 来自计数)
2. { { "Peru": 1, "Argentina": 2, "Brazil": 2, "Iran": 2, "China": 2, "Italy": 1, "France": 2, "United States": 1, "Canada": 1 } } (有效, 无需改进) (2 个错误, 来自计数)
3. { { "Peru": 1, "Argentina": 2, "Brazil": 2, "Iran": 2, "China": 2, "Italy": 1, "France": 2, "United States": 1, "Canada": 1 } } (有效, 无需改进) (2 个错误, 来自计数)

表 27: 将文本拆成 4 段的关键词计数示例 (GoT4): 步骤 3b (提示/响应)

步骤 3b – 提示:

<Instruction> 将以下两个字典合并成一个字典, 每个字典都包含一段文本中各国的出现频次。只需将每个国家的频次相加; 如果某个国家未出现在其中一个字典中, 则将它加入最终字典, 并使用另一个字典中的频次。

只输出最终合并后的字典, 不要输出额外文字或思考! </Instruction>

<Approach>

按以下步骤将两个字典合并成一个字典:

1. 创建一个新字典, 用于存储合并后的频次。
2. 遍历第一个字典的键, 将每个国家及其频次加入新字典。
3. 遍历第二个字典的键, 将每个国家及其频次加入新字典; 如果该国已经存在, 就将其频次加到现有值上。

</Approach>

将以下两个字典合并成一个字典:

```
{{ "United States": 1, "Canada": 1, "Australia": 2, "New Zealand": 2, "Ukraine": 1, "Poland": 1 }}
```

```
{{ "Ukraine": 1, "Poland": 1, "North Korea": 1, "South Korea": 2, "Argentina": 1 }}
```

合并后的输出:

步骤 3b – 3 个响应:

1.

```
{{ "United States": 1, "Canada": 1, "Australia": 2, "New Zealand": 2, "Ukraine": 2, "Poland": 2, "North Korea": 1, "South Korea": 2, "Argentina": 1 }}
```

 (有效, 无需改进) (完全正确)
 2.

```
{{ "United States": 1, "Canada": 1, "Australia": 2, "New Zealand": 2, "Ukraine": 2, "Poland": 2, "North Korea": 1, "South Korea": 2, "Argentina": 1 }}
```

 (有效, 无需改进) (完全正确)
 3.

```
{{ "United States": 1, "Canada": 1, "Australia": 2, "New Zealand": 2, "Ukraine": 2, "Poland": 2, "North Korea": 1, "South Korea": 2, "Argentina": 1 }}
```

 (有效, 无需改进) (完全正确)
-

表 28: 将文本拆成 4 段的关键词计数示例 (GoT4): 步骤 3c (提示/响应)

步骤 3c – 提示:

<Instruction> 将以下两个字典合并成一个字典, 每个字典都包含一段文本中各国的出现频次。只需将每个国家的频次相加; 如果某个国家未出现在其中一个字典中, 则将它加入最终字典, 并使用另一个字典中的频次。

只输出最终合并后的字典, 不要输出额外文字或思考! </Instruction>

<Approach>

按以下步骤将两个字典合并成一个字典:

1. 创建一个新字典, 用于存储合并后的频次。
2. 遍历第一个字典的键, 将每个国家及其频次加入新字典。
3. 遍历第二个字典的键, 将每个国家及其频次加入新字典; 如果该国已经存在, 就将其频次加到现有值上。

</Approach>

将以下两个字典合并成一个字典:

```
{{ "Peru": 1, "Argentina": 2, "Brazil": 2, "Iran": 2, "China": 2, "Italy": 1, "France": 2, "United States": 1, "Canada": 1 }}
```

```
{{ "United States": 1, "Canada": 1, "Australia": 2, "New Zealand": 2, "Ukraine": 2, "Poland": 2, "North Korea": 1, "South Korea": 2, "Argentina": 1 }}
```

合并后的输出:

步骤 3c – 3 个响应:

1.

```
{{ "Peru": 1, "Argentina": 3, "Brazil": 2, "Iran": 2, "China": 2, "Italy": 1, "France": 2, "United States": 2, "Canada": 2, "Australia": 2, "New Zealand": 2, "Ukraine": 2, "Poland": 2, "North Korea": 1, "South Korea": 2 }}
```

 (有效, 无需改进) (2 个错误, 来自计数)
2.

```
{{ "Peru": 1, "Argentina": 3, "Brazil": 2, "Iran": 2, "China": 2, "Italy": 1, "France": 2, "United States": 2, "Canada": 2, "Australia": 2, "New Zealand": 2, "Ukraine": 2, "Poland": 2, "North Korea": 1, "South Korea": 2 }}
```

 (有效, 无需改进) (2 个错误, 来自计数)
3.

```
{{ "Peru": 1, "Argentina": 3, "Brazil": 2, "Iran": 2, "China": 2, "Italy": 1, "France": 2, "United States": 2, "Canada": 2, "Australia": 2, "New Zealand": 2, "Ukraine": 2, "Poland": 2, "North Korea": 1, "South Korea": 2 }}
```

 (有效, 无需改进) (2 个错误, 来自计数)

最终结果 (2 个错误):

```
{{ "Peru": 1, "Argentina": 3, "Brazil": 2, "Iran": 2, "China": 2, "Italy": 1, "France": 2, "United States": 2, "Canada": 2, "Australia": 2, "New Zealand": 2, "Ukraine": 2, "Poland": 2, "North Korea": 1, "South Korea": 2 }}
```

E 提示示例——文档合并

这里只展示文档合并任务的 GoT 提示，因为 GoT2 的唯一区别是分 2 步而非 1 步合并 4 份 NDA。在文档合并任务中，我们使用四种不同的操作与 LLM 交互，每种操作都有相应提示。首先是**生成 (Generate)** 操作，使用 *merge_prompt* 指示 LLM 将 4 份 NDA 合并为 1 份。其次是**评分 (Score)** 操作，使用 *score_prompt* 指示 LLM 对给定的合并后 NDA 评分。接着，**聚合 (Aggregate)** 操作使用 *aggregate_prompt* 指示 LLM 将多次合并尝试的结果聚合为一个更好的结果。最后，**改进 (Improve)** 操作使用 *improve_prompt* 指示 LLM 改进一份已经合并的 NDA。

首先展示提示模板（表 29 至表 30），它们用于在运行时动态生成适当的提示。随后展示一次完整合并过程与 LLM 的交互（表 31 至表 49）。这里不在每次提示/响应中展示每份输入或生成的 NDA，而是在表 31 至表 33 中展示 4 份输入 NDA，在表 49 中展示最终合并后的 NDA。此外，由于评分也由 LLM 完成，我们展示表现最佳的合并后 NDA 的评分交互（表 39 至表 40，以及表 47 至表 48）。最后，大多数响应仅展示数行，因为余下内容无法提供更多洞见，而完整展示会占据多页。感兴趣的读者可查阅对应代码仓库中的结果²，获取完整日志和更多示例。

²<https://github.com/spcl/graph-of-thoughts>

表 29: 文档合并任务的提示模板；单层花括号中的参数会在运行时被替换。

merge_prompt: 将以下 4 份保密协议（NDA）文档 <Doc1> 至 <Doc4> 合并成一份 NDA，尽可能保留信息并减少冗余。只在 <Merged> 与 </Merged> 标签之间输出生成的 NDA，不要输出额外文字。

以下是 NDA <Doc1> 至 <Doc4>：

<Doc1> {doc1} </Doc1>

<Doc2> {doc2} </Doc2>

<Doc3> {doc3} </Doc3>

<Doc4> {doc4} </Doc4>

score_prompt: 以下 NDA <S> 合并了 NDA <Doc1> 至 <Doc4>。

请为合并后的 NDA <S> 评分：一方面评估其自身包含多少冗余信息，不以原始 NDA 为参照；另一方面评估它保留了原始 NDA 中的多少信息。

冗余评分为 10 表示完全没有冗余信息；评分为 0 表示至少一半的信息是冗余的（即所有内容都至少被提及两次）。

信息保留评分为 10 表示保留了原始 NDA 中的全部信息；评分为 0 表示没有保留任何信息。

可以解释评分理由，但最终的冗余评分应置于 <Redundancy> 与 </Redundancy> 标签之间，最终的信息保留评分应置于 <Retained> 与 </Retained> 标签之间；这些标签内均不得包含额外文字。

以下是 NDA <Doc1> 至 <Doc4>：

<Doc1> {doc1} </Doc1>

<Doc2> {doc2} </Doc2>

<Doc3> {doc3} </Doc3>

<Doc4> {doc4} </Doc4>

以下是合并后的 NDA <S>：

<S> {s} </S>

aggregate_prompt: 以下 NDA <S1> 至 <S{num_ndas_summaries}> 中，每一份都合并了最初的 NDA <Doc1> 至 <Doc4>。

将已合并的 NDA <S1> 至 <S{num_ndas_summaries}> 合成为一份新 NDA，尽可能发挥它们的优势、保留整体信息，并减少冗余。

只在 <Merged> 与 </Merged> 标签之间输出新的 NDA，不要输出额外文字。

以下是原始 NDA <Doc1> 至 <Doc4>：

<Doc1> {doc1} </Doc1>

<Doc2> {doc2} </Doc2>

<Doc3> {doc3} </Doc3>

<Doc4> {doc4} </Doc4>

以下是已合并的 NDA <S1> 至 <S{num_ndas_summaries}>：

<S1> {s1} </S1>

...

<S{num_ndas_summaries}> {s{num_ndas_summaries}} </S{num_ndas_summaries}>

表 30: 文档合并任务的提示模板（续）；单层花括号中的参数会在运行时被替换。

improve_prompt: 以下 NDA <S> 合并了最初的 NDA <Doc1> 至 <Doc4>。

请通过补充更多信息并删除冗余内容，改进合并后的 NDA <S>。只输出改进后的 NDA，将其置于 <Merged> 与 </Merged> 标签之间，不要输出额外文字。

以下是 NDA <Doc1> 至 <Doc4>：

```
<Doc1> {doc1} </Doc1>
<Doc2> {doc2} </Doc2>
<Doc3> {doc3} </Doc3>
<Doc4> {doc4} </Doc4>
```

以下是合并后的 NDA <S>：

```
<S> {s} </S>
```

表 31: 输入的 NDA 1 和 NDA 2

```
<Doc1>
保密协议（NDA）
1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 签订的协议。
2. 为 [specific project or purpose] 共享信息。
3. “保密信息”包括一切可能具有商业价值的信息，具体包括软件开发策略、流程和内部研究成果。
4. 接收方有义务保护保密信息，仅为所披露的目的使用该信息，并且未经同意不得披露。
5. 违约后果包括禁令救济、其他救济，以及每次违约支付 $200,000。
6. 本协议适用于各方及其继承人和受让人。它包含全部相关约定，未予执行不代表放弃权利。
7. 本协议受 [State] 的法律管辖。
8. 由 [Your Company Name] 和 [Recipient Name] 于上述日期签署。
</Doc1>
```

```
<Doc2>
保密协议（NDA）
本 NDA 自 [Effective Date] 起生效，涉及 [Your Company Name]（“披露方”）及 [Recipient Name]（“接收方”）。
1. 目的：披露方将为 [Purpose] 向接收方披露与 [Topic of Research] 有关的保密信息。
2. 保密信息：指披露方向接收方提供的一切非公开报告、数据、设计和其他材料。
3. 接收方的义务：
  a. 仅为约定目的使用、复制或分发保密信息。
  b. 仅允许必要的相关方接触该信息，并确保其严格遵守保密要求。
  c. 在收到要求或协议期满时，返还或销毁全部保密信息。
4. 例外：若信息已为接收方所知、已公开，或由接收方独立开发，则不将其归为保密信息。
5. 竞业限制：接收方在协议存续期间以及协议终止后一年内，不得从事与披露方竞争的业务。
6. 期限与终止：本协议有效期为 [e.g., "two years"]，除非提前 [e.g., "30 days"] 发出书面通知而终止。接收方的保密和竞业限制义务在终止后继续有效。
7. 一般条款：
  a. 适用法律：适用 [Your State] 的法律。
  b. 修订：仅以书面作出并经双方签署的修订有效。
  c. 完整协议：本合同取代此前的相关协议。
于生效日期由 [Your Company Name]（披露方）和 [Recipient Name]（接收方）签署。
</Doc2>
```

表 32: 输入的 NDA 3

<Doc3>

保密与不披露协议

涉及的主体:

于 [Date] 生效, 协议双方为在 [State] 设立的 [AquaBlue Innovations] 和注册实体 [PineTree Solutions]。

目标:

在商谈潜在技术联盟期间保护机密数据。

受保护信息的具体内容:

具体包括:

- a. 系统设计和架构示意图。
- b. 专有计算算法。

接收者的义务:

- a. 采用最佳实践, 严格遵守不披露要求。
- b. 仅为上述目的使用。
- c. 未经明确授权不得公开。

违反协议的后果:

每次违规支付 \$280,000, 并可能面临法律诉讼。

一般约定:

约束双方及其任何继承人, 并构成全部约定。

法律依据:

依据 [State] 的法律框架管辖。

签署证明:

于 [Date] 正式签署。

[AquaBlue Innovations] [PineTree Solutions]

</Doc3>

表 33: 输入的 NDA 4

<Doc4>
 保密与披露协议
 缔约方：
 日期为 [Date]，由位于 [State] 的公司 [AquaBlue Innovations] 与持牌组织 [PineTree Solutions] 签订。
 目的：
 在就技术合作开展对话期间，保护专有见解。
 敏感数据的类别：
 包括：
 a. 内部软件蓝图。
 b. 等待授予专利的知识产权。
 接收方的承诺：
 a. 保持机密性，并确保数据完整性。
 b. 严格仅用于合作事项。
 c. 未经事先一致同意不得披露。
 不遵守协议的后果：
 每次违规罚款 \$295,000，并可寻求法律救济。
 总体条款：
 对签署方及其继承人具有法律约束力；构成完整且唯一的协议。
 法律立场：
 受 [State] 法律管辖。
 确认：
 于 [Date] 确认并签署。
 [AquaBlue Innovations] [PineTree Solutions]
 </Doc4>

表 34: 合并 4 份示例 NDA：执行计划 (GoO)

GoO:

1. 将 4 份 NDA 合并成一份，共尝试 5 次；为每次尝试评分，保留最好的 3 个结果
 2. 将各次合并尝试的结果聚合为一份，共尝试 5 次；为每次聚合尝试评分，保留全部尝试中的最佳结果（包括步骤 1 的结果）
 3. 改进合并后的 NDA，共尝试 10 次；逐一评分，保留最佳结果
-

表 35: 合并 4 份示例 NDA：步骤 1 (提示)

将以下 4 份保密协议（NDA）文档 <Doc1> 至 <Doc4> 合并成一份 NDA，尽可能保留信息并减少冗余。只在 <Merged> 与 </Merged> 标签之间输出生成的 NDA，不要输出额外文字。

以下是 NDA <Doc1> 至 <Doc4>：

<Doc1>

保密协议（NDA）

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 签订的协议。
2. 为 [specific project or purpose] 共享信息。

... (省略, 见表 31)

</Doc1>

<Doc2>

保密协议（NDA）

本 NDA 自 [Effective Date] 起生效，涉及 [Your Company Name]（“披露方”）及 [Recipient Name]（“接收方”）。

1. 目的：披露方将为 [Purpose] 向接收方披露与 [Topic of Research] 有关的保密信息。

... (省略, 见表 31)

</Doc2>

<Doc3>

保密与不披露协议

涉及的主体：

于 [Date] 生效，协议双方为在 [State] 设立的 [AquaBlue Innovations] 和注册实体 [PineTree Solutions]。

目标：

在商谈潜在的 ... (省略, 见表 32)

</Doc3>

<Doc4>

保密与披露协议

缔约方：

日期为 [Date]，由位于 [State] 的公司 [AquaBlue Innovations] 与持牌组织 [PineTree Solutions] 签订。

目的：

在对话期间保护专有见解 ... (省略, 见表 33)

</Doc4>

表 36: 合并 4 份示例 NDA；仅在表 37 至表 38 中完整展示最高分响应：步骤 1（响应）

由于响应被截短，它们看起来相同，但查看完整生成的摘要时，可以发现细微差别。差异很小，也体现为十分接近的评分（最佳响应的评分见表 39 至表 40）。

响应 (1/5):

<Merged>

保密协议（NDA）

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 签订的协议。
 2. 为 [specific project or purpose] 共享信息。
- ... （省略）

评分: 6.60

响应 (2/5):

<Merged>

保密协议（NDA）

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 签订的协议。
 2. 为 [specific project or purpose] 共享信息。
- ... （省略，见表 37 - 38）

评分: 6.87

响应 (3/5):

<Merged>

保密协议（NDA）

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 签订的协议。
 2. 为 [specific project or purpose] 共享信息。
- ... （省略）

评分: 6.60

响应 (4/5):

<Merged>

保密协议（NDA）

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 签订的协议。
 2. 为 [specific project or purpose] 共享信息。
- ... （省略）

评分: 5.78

响应 (5/5):

<Merged>

保密协议（NDA）

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 签订的协议。
 2. 为 [specific project or purpose] 共享信息。
- ... （省略）

评分: 6.50

表 37: 合并 4 份示例 NDA: 步骤 1 (最高分响应, 第 1 部分)

保密协议 (NDA)

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 签订的协议。
2. 为 [specific project or purpose] 共享信息。
3. “保密信息”包括一切可能具有商业价值的信息, 具体包括软件开发策略、流程和内部研究成果。
4. 接收方有义务保护保密信息, 仅为所披露的目的使用该信息, 并且未经同意不得披露。
5. 违约后果包括禁令救济、其他救济, 以及每次违约支付 \$200,000。
6. 本协议适用于各方及其继承人和受让人。它包含全部相关约定, 未予执行不代表放弃权利。
7. 本协议受 [State] 的法律管辖。
8. 由 [Your Company Name] 和 [Recipient Name] 于上述日期签署。

本 NDA 自 [Effective Date] 起生效, 涉及 [Your Company Name] (“披露方”) 及 [Recipient Name] (“接收方”)。

目的: 披露方将为 [Purpose] 向接收方披露与 [Topic of Research] 有关的保密信息。

保密信息: 指披露方向接收方提供的一切非公开报告、数据、设计和其他材料。

接收方的义务:

- a. 仅为约定目的使用、复制或分发保密信息。
- b. 仅允许必要的相关方接触该信息, 并确保其严格遵守保密要求。
- c. 在收到要求或协议期满时, 返还或销毁全部保密信息。

例外: 若信息已为接收方所知、已公开, 或由接收方独立开发, 则不将其归为保密信息。

竞业限制: 接收方在协议存续期间以及协议终止后一年内, 不得从事与披露方竞争的业务。

期限与终止: 本协议有效期为 [e.g., "two years"], 除非提前 [e.g., "30 days"] 发出书面通知而终止。接收方的保密和竞业限制义务在终止后继续有效。

一般条款:

- a. 适用法律: 适用 [Your State] 的法律。
- b. 修订: 仅以书面作出并经双方签署的修订有效。
- c. 完整协议: 本合同取代此前的相关协议。

于生效日期由 [Your Company Name] (披露方) 和 [Recipient Name] (接收方) 签署。

涉及的主体:

于 [Date] 生效, 协议双方为在 [State] 设立的 [AquaBlue Innovations] 和注册实体 [PineTree Solutions]。

目标:

在商谈潜在技术联盟期间保护机密数据。

表 38: 合并 4 份示例 NDA: 步骤 1 (最高分响应, 第 2 部分)

受保护信息的具体内容:

具体包括:

- a. 系统设计和架构示意图。
- b. 专有计算算法。

接收者的义务:

- a. 采用最佳实践, 严格遵守不披露要求。
- b. 仅为上述目的使用。
- c. 未经明确授权不得公开。

违反协议的后果:

每次违规支付 \$280,000, 并可能面临法律诉讼。

一般约定:

约束双方及其任何继承人, 并构成全部约定。

法律依据:

依据 [State] 的法律框架管辖。

签署证明:

于 [Date] 正式签署。

缔约方:

日期为 [Date], 由位于 [State] 的公司 [AquaBlue Innovations] 与持牌组织 [PineTree Solutions] 签订。

目的:

在就技术合作开展对话期间, 保护专有见解。

敏感数据的类别:

包括:

- a. 内部软件蓝图。
- b. 等待授予专利的知识产权。

接收方的承诺:

- a. 保持机密性, 并确保数据完整性。
- b. 严格仅用于合作事项。
- c. 未经事先一致同意不得披露。

不遵守协议的后果:

每次违规罚款 \$295,000, 并可寻求法律救济。

总体条款:

对签署方及其继承人具有法律约束力; 构成完整且唯一的协议。

法律立场:

受 [State] 法律管辖。

确认:

于 [Date] 确认并签署。

表 39: 合并 4 份示例 NDA: 步骤 1 (评分示例——提示)

请为合并后的 NDA <S> 评分：一方面评估其自身包含多少冗余信息，不以原始 NDA 为参照；另一方面评估它保留了原始 NDA 中的多少信息。

冗余评分为 10 表示完全没有冗余信息；评分为 0 表示至少一半的信息是冗余的（即所有内容都至少被提及两次）。

信息保留评分为 10 表示保留了原始 NDA 中的全部信息；评分为 0 表示没有保留任何信息。

可以解释评分理由，但最终的冗余评分应置于 <Redundancy> 与 </Redundancy> 标签之间，最终的信息保留评分应置于 <Retained> 与 </Retained> 标签之间；这些标签内均不得包含额外文字。

以下是 NDA <Doc1> 至 <Doc4>：

<Doc1>

保密协议（NDA）

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 签订的协议。

... (省略, 见表 31)

</Doc1>

<Doc2>

保密协议（NDA）

本 NDA 自 [Effective Date] 起生效，涉及 [Your Company Name]（“披露方”）及 [Recipient Name]（“接收方”）。

... (省略, 见表 31)

</Doc2>

<Doc3>

保密与不披露协议

涉及的主体：

于 [Date] 生效，协议双方为在 [State] 设立的 [AquaBlue Innovations] 和 ... (省略, 见表 32)

</Doc3>

<Doc4>

保密与披露协议

缔约方：

日期为 [Date]，由位于 [State] 的公司 [AquaBlue Innovations] 与 ... (省略, 见表 33)

</Doc4>

以下是汇总后的 NDA <S>：

<S>

保密协议（NDA）

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 签订的协议。

2. 为 [specific project or purpose] 共享信息。

... (省略, 见表 37-38)

</S>

表 40: 合并 4 份示例 NDA：步骤 1 (评分示例——响应)

响应 (1/3):

<Redundancy>5</Redundancy>

<Retained>10</Retained>

评分理由：

从冗余角度看，各份 NDA 所含信息存在一定重叠，例如“保密信息”的定义和接收方的义务。不过，每份 NDA 也含有一些独特细节，例如共享信息的具体目的和违约处罚。总体而言，合并后的 NDA 存在中等程度的冗余。

从信息保留角度看，原始 NDA 的全部关键要素都包含在合并后的 NDA 中，包括各方之间的约定、信息共享目的、保密信息的定义、接收方的义务、保密信息的例外、竞业限制条款、协议的期限与终止，以及一般条款。因此，合并后的 NDA 保留了原始 NDA 的全部重要信息。

响应 (2/3):

<Redundancy>8</Redundancy>

<Retained>10</Retained>

评分理由：

合并后的 NDA <S> 包含来自原始 NDA 的多个冗余部分。例如，各方之间的约定、信息共享目的等开篇部分重复出现了多次。此外，有关保密信息定义、接收方义务、例外、竞业限制、期限与终止，以及一般条款的部分也被重复纳入。

不过，从信息保留角度看，合并后的 NDA 保留了原始 NDA 的全部关键要素，包括相关主体、目标、受保护信息的具体内容、接收方义务、违反协议的后果、一般约定、法律依据及签署证明。总体而言，虽然合并后的 NDA 存在冗余，但它成功保留了原始 NDA 中全部必要的信息。

响应 (3/3):

<Redundancy>3</Redundancy>

合并后的 NDA 存在一些冗余。原始 NDA 中有三个部分的信息相互重叠，包括：

1. 各方之间的约定（Doc1 第 1 节和 Doc2 的开头）
2. 信息共享以及 NDA 的目的（Doc1 和 Doc2 中的第 2 节）
3. 适用法律、修订等一般条款（Doc1 和 Doc2 中的第 7 节）

<Retained>9</Retained>

合并后的 NDA 保留了原始 NDA 中的大量信息，包括全部关键条款和约定，例如保密信息的定义、接收方义务、例外、竞业限制以及一般条款。仅有少数次要细节未被保留，例如具体处罚和罚款金额。总体而言，合并后的 NDA 有效地结合了原始 NDA 的核心要素。

说明：提供最终评分时，请记得删除评分标签之间的解释文字。

最终总体评分（各项平均值的调和平均）:

6.87

表 41: 合并 4 份示例 NDA；S1-S3 是上一步评分最高的三个合并结果：**步骤 2（提示）**

以下 NDA <S1> 至 <S3> 中，每一份都合并了最初的 NDA <Doc1> 至 <Doc4>。
 将已合并的 NDA <S1> 至 <S3> 合成为一份新 NDA，尽可能发挥它们的优势、保留整体信息，并减少冗余。
 只在 <Merged> 与 </Merged> 标签之间输出新的 NDA，不要输出额外文字。

以下是原始 NDA <Doc1> 至 <Doc4>：

```
<Doc1>
... (省略, 见表 31)
</Doc1>
<Doc2>
... (省略, 见表 31)
</Doc2>
<Doc3>
... (省略, 见表 32)
</Doc3>
<Doc4>
... (省略, 见表 33)
</Doc4>
```

以下是已合并的 NDA <S1> 至 <S3>：

```
<S1>
保密协议（NDA）
1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 签订的协议。
... (省略)
</S1>
<S2>
保密协议（NDA）
1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 签订的协议。
... (省略, 见表 37-38)
</S2>
<S3>
保密协议（NDA）
1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 签订的协议。
... (省略)
</S3>
```

表 42: 合并 4 份示例 NDA: **步骤 2 (响应)**

在本次运行中，没有一个响应超过先前的最佳结果，因此它们都会在下一步被丢弃。基于这一原因，加之它们篇幅较长，这里不完整展示这些响应，也不展示评分过程。

响应 (1/5):

<Merged>

保密协议 (NDA)

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 签订的协议。
2. 为 [specific project or purpose] 共享信息。

... (省略)

评分: 6.50

响应 (2/5):

<Merged>

保密协议 (NDA)

本 NDA 自 [Effective Date] 起生效，涉及 [Your Company Name] (“披露方”) 及 [Recipient Name] (“接收方”)。

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 签订的协议。

... (省略)

评分: 5.62

响应 (3/5):

<Merged>

保密协议 (NDA)

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 签订的协议。
2. 为 [specific project or purpose] 共享信息。

... (省略)

评分: 6.60

响应 (4/5):

<Merged>

保密协议 (NDA)

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 签订的协议。
2. 为 [specific project or purpose] 共享信息。

... (省略)

评分: 6.15

响应 (5/5):

<Merged>

保密协议 (NDA)

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 签订的协议。
2. 为 [specific project or purpose] 共享信息。

... (省略)

评分: 6.22

表 43: 合并 4 份示例 NDA: 步骤 3 (提示)

以下 NDA <S> 合并了最初的 NDA <Doc1> 至 <Doc4>。

请通过补充更多信息并删除冗余内容, 改进合并后的 NDA <S>。只输出改进后的 NDA, 将其置于 <Merged> 与 </Merged> 标签之间, 不要输出额外文字。

以下是 NDA <Doc1> 至 <Doc4>:

<Doc1>

保密协议 (NDA)

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 签订的协议。
2. 为 [specific project or purpose] 共享信息。

... (省略, 见表 31)

</Doc1>

<Doc2>

保密协议 (NDA)

本 NDA 自 [Effective Date] 起生效, 涉及 [Your Company Name] (“披露方”) 及 [Recipient Name] (“接收方”)。

1. 目的: 披露方将为 [Purpose] 向接收方披露与 [Topic of Research] 有关的保密信息。

... (省略, 见表 31)

</Doc2>

<Doc3>

保密与不披露协议

涉及的主体:

于 [Date] 生效, 协议双方为在 [State] 设立的 [AquaBlue Innovations] 和注册实体 [PineTree Solutions]。

目标:

在商谈潜在的 ... (省略, 见表 32)

</Doc3>

<Doc4>

保密与披露协议

缔约方:

日期为 [Date], 由位于 [State] 的公司 [AquaBlue Innovations] 与持牌组织 [PineTree Solutions] 签订。

目的:

在对话期间保护专有见解 ... (省略, 见表 33)

</Doc4>

以下是汇总后的 NDA <S>:

<S>

保密协议 (NDA)

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 签订的协议。
2. 为 [specific project or purpose] 共享信息。

... (省略, 见表 37-38)

</S>

表 44: 合并 4 份示例 NDA；仅在表 46 中完整展示最高分响应：步骤 3（响应，第 1 部分）

响应 (1/10):

<Merged>

保密协议 (NDA)

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 签订的协议。
2. 为 [specific project or purpose] 共享信息。

... (省略)

评分: 6.35

响应 (2/10):

<Merged>

保密协议 (NDA)

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 签订的协议。
2. 为 [specific project or purpose] 共享信息。

... (省略)

评分: 6.35

响应 (3/10):

<Merged>

保密协议 (NDA)

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 为 [specific project or purpose] 签订的协议。
2. “保密信息”包括一切可能具有商业价值的信息，具体包括软件开发策略、流程、内部研究成果、系统设计、架构示意图、专有计算算法、内部软件蓝图和等待授予专利的知识产权。

... (省略)

评分: 6.87

响应 (4/10):

<Merged>

保密协议 (NDA)

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 为 [specific project or purpose] 签订的协议。
2. “保密信息”包括一切可能具有商业价值的信息，例如软件开发策略、流程、内部研究成果、系统设计、架构示意图、专有计算算法、内部软件蓝图和等待授予专利的知识产权。

... (省略, 见表 46))

评分: 7.78

响应 (5/10):

<Merged>

保密协议 (NDA)

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 签订的协议。
2. 为 [specific project or purpose] 共享信息。

... (省略)

评分: 6.43

表 45: 合并 4 份示例 NDA；仅在表 46 中完整展示最高分响应：步骤 3（响应，第 2 部分）

响应 (6/10):

<Merged>

保密协议 (NDA)

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 签订的协议。
2. 为 [specific project or purpose] 共享信息。

... (省略)

评分: 6.35

响应 (7/10):

<Merged>

保密协议 (NDA)

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 为 [specific project or purpose] 签订的协议。
2. “保密信息”包括一切可能具有商业价值的信息，具体包括软件开发策略、流程、内部研究成果、系统设计、架构示意图、专有计算算法、内部软件蓝图和等待授予专利的知识产权。

... (省略)

评分: 6.70

响应 (8/10):

<Merged>

保密协议 (NDA)

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 为 [specific project or purpose] 签订的协议。
2. “保密信息”包括可能具有商业价值的信息，例如软件开发策略、流程、内部研究成果，以及披露方提供的非公开报告、数据、设计和其他材料。

... (省略)

评分: 5.33

响应 (9/10):

<Merged>

保密协议 (NDA)

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 签订的协议。
2. 为 [specific project or purpose] 共享信息，包括 [Topic of Research]。

... (省略)

评分: 7.30

响应 (10/10):

<Merged>

保密协议 (NDA)

协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 签订的协议。

为 [specific project or purpose] 共享信息。

... (省略)

评分: 6.40

表 46: 合并 4 份示例 NDA: 步骤 3 (最高分响应)

保密协议 (NDA)

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 为 [specific project or purpose] 签订的协议。
2. “保密信息”包括一切可能具有商业价值的信息, 例如软件开发策略、流程、内部研究成果、系统设计、架构示意图、专有计算算法、内部软件蓝图和等待授予专利的知识产权。
3. 接收方有义务保护保密信息, 仅为所披露的目的使用该信息, 并且未经同意不得披露。
4. 违约后果包括禁令救济、其他救济, 以及每次违约的罚款 (例如, 视协议而定, 每次违规支付 \$200,000 或 \$280,000)。
5. 本协议适用于各方及其继承人和受让人。它包含全部相关约定, 未予执行不代表放弃权利。
6. 本协议受 [State] 的法律管辖, 在指定期限内有效 (例如两年), 除非通过事先书面通知提前终止 (例如提前 30 天)。
7. 接收方同意在协议存续期间以及终止后的指定期限内 (例如一年), 不从事与披露方竞争的业务。
8. 本协议仅能通过双方签署的书面文件修订, 并取代此前的相关协议。
9. 本协议具有法律效力, 约束签署方及其继承人。
10. 本协议受 [State] 的法律管辖。

于生效日期由 [Your Company Name] (披露方)、[Recipient Name] (接收方) 以及其他相关方签署。

表 47: 合并 4 份示例 NDA: 步骤 3 (评分示例——提示)

请为合并后的 NDA <S> 评分: 一方面评估其自身包含多少冗余信息, 不以原始 NDA 为参照; 另一方面评估它保留了原始 NDA 中的多少信息。

冗余评分为 10 表示完全没有冗余信息; 评分为 0 表示至少一半的信息是冗余的 (即所有内容都至少被提及两次)。信息保留评分为 10 表示保留了原始 NDA 中的全部信息; 评分为 0 表示没有保留任何信息。

可以解释评分理由, 但最终的冗余评分应置于 <Redundancy> 与 </Redundancy> 标签之间, 最终的信息保留评分应置于 <Retained> 与 </Retained> 标签之间; 这些标签内均不得包含额外文字。

以下是 NDA <Doc1> 至 <Doc4>:

<Doc1>

保密协议 (NDA)

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 签订的协议。

... (省略, 见表 31)

</Doc1>

<Doc2>

保密协议 (NDA)

本 NDA 自 [Effective Date] 起生效, 涉及 [Your Company Name] (“披露方”) 及 [Recipient Name] (“接收方”)。

... (省略, 见表 31)

</Doc2>

<Doc3>

保密与不披露协议

涉及的主体:

于 [Date] 生效, 协议双方为在 [State] 设立的 [AquaBlue Innovations] 和 ... (省略, 见表 32)

</Doc3>

<Doc4>

保密与披露协议

缔约方:

日期为 [Date], 由位于 [State] 的公司 [AquaBlue Innovations] 与 ... (省略, 见表 33)

</Doc4>

以下是汇总后的 NDA <S>:

<S>

保密协议 (NDA)

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 为 [specific project or purpose] 签订的协议。

2. “保密信息”包括一切可能具有商业价值的信息, 例如软件开发策略、流程、内部研究成果、系统设计、架构示意图、专有计算算法、内部软件蓝图和等待授予专利的知识产权。

... (省略, 见表 46)

</S>

表 48: 合并 4 份示例 NDA：步骤 3 (评分示例——响应)

响应 (1/3):

<Redundancy>5</Redundancy>

<Retained>8</Retained>

评分理由:

合并后的 NDA <S> 保留了原始 NDA 中的大部分信息，将相关条款和约定组合成一份全面的协议。不过，类似信息多次出现，例如“保密信息”的定义及接收方义务，因此仍存在一定冗余。此外，原始 NDA 的某些具体细节未被纳入合并后的 NDA，例如 Doc2 中的例外条件，以及 Doc3 和 Doc4 中的竞业限制条款。尽管如此，合并后的 NDA 仍抓住了原始 NDA 的要旨，并包含保密协议所需的全部关键信息。

响应 (2/3):

<Redundancy>8</Redundancy>

<Retained>10</Retained>

评分理由:

合并后的 NDA <S> 保留了原始 NDA 的全部关键信息，包括协议目的、保密信息的定义、接收方义务、违约处罚、对各方及继承人的适用性、适用法律和签署要求。

其中存在一些冗余信息，例如同时从 Doc3 和 Doc4 纳入系统设计、架构示意图、专有计算算法、内部软件蓝图，以及等待授予专利的知识产权。不过，这些冗余很少，不会显著影响 NDA 的整体实用性。

总体而言，合并后的 NDA <S> 在减少冗余的同时保留了全部重要信息，能够十分有效地传达协议所需的条款和义务。因此，冗余评分为 8，信息保留评分为 10。

响应 (3/3):

<Redundancy>7</Redundancy>

<Retained>10</Retained>

评分理由:

合并后的 NDA <S> 包含一些冗余信息，例如“保密信息”的定义和接收方义务。不过，它保留了原始 NDA 的全部重要信息，包括协议目的、保密信息的具体类型、违约处罚、对各方及继承人的适用性、适用法律，以及竞业限制、修订和终止条款。此外，它还纳入了有关协议有效期及不遵守协议时可寻求法律救济的补充信息。因此，合并后的 NDA <S> 的冗余评分为 7，信息保留评分为 10。

最终总体评分（各项平均值的调和平均）:

7.78

表 49: 合并 4 份示例 NDA：最终合并后的 NDA

保密协议（NDA）

1. 协议由 [Your Company Name] 与 [Recipient Name] 于 [Date] 为 [specific project or purpose] 签订的协议。
2. “保密信息”包括一切可能具有商业价值的信息，例如软件开发策略、流程、内部研究成果、系统设计、架构示意图、专有计算算法、内部软件蓝图和等待授予专利的知识产权。
3. 接收方有义务保护保密信息，仅为所披露的目的使用该信息，并且未经同意不得披露。
4. 违约后果包括禁令救济、其他救济，以及每次违约的罚款（例如，视协议而定，每次违规支付 \$200,000 或 \$280,000）。
5. 本协议适用于各方及其继承人和受让人。它包含全部相关约定，未予执行不代表放弃权利。
6. 本协议受 [State] 的法律管辖，在指定期限内有效（例如两年），除非通过事先书面通知提前终止（例如提前 30 天）。
7. 接收方同意在协议存续期间以及终止后的指定期限内（例如一年），不从事与披露方竞争的业务。
8. 本协议仅能通过双方签署的书面文件修订，并取代此前的相关协议。
9. 本协议具有法律效力，约束签署方及其继承人。
10. 本协议受 [State] 的法律管辖。

于生效日期由 [Your Company Name]（披露方）、[Recipient Name]（接收方）以及其他相关方签署。

F 评估——GoT 配置

下面列出为求解集合求交及排序用例而为 GoT 配置的具体操作。

算法 1: GoT 用于 32 元素集合求交用例的配置

```
1 Generate(k=1) # 将第二个集合分成两半，每半 16 个元素
2 foreach subset:
3   Generate(k=5) # 计算该子集与第一个输入集合的交集
4   Score(k=1) # 在本地为交集子集评分
5   KeepBestN(1) # 保留最佳交集子集
6 Aggregate(10) # 合并两个交集子集
7 Score(k=1) # 在本地为求交结果集合评分
8 KeepBestN(1) # 保留最佳结果
9 GroundTruth() # 与预先计算的结果比较
```

算法 2: GoT 用于 64 元素集合求交用例的配置

```
1 Generate(k=1) # 将第二个集合分成四部分，每部分 16 个元素
2 foreach subset:
3   Generate(k=5) # 计算该子集与第一个输入集合的交集
4   Score(k=1) # 在本地为交集子集评分
5   KeepBestN(1) # 保留最佳交集子集
6 merge step 1:
7   Aggregate(10) # 合并交集子集 1 和 2
8   Score(k=1) # 在本地为求交结果集合评分
9   KeepBestN(1) # 保留最佳结果
10 merge step 2:
11   Aggregate(10) # 合并交集子集 3 和 4
12   Score(k=1) # 在本地为求交结果集合评分
13   KeepBestN(1) # 保留最佳结果
14 final merge:
15   Aggregate(10) # 合并步骤 1 和 2 产生的中间交集子集
16   Score(k=1) # 在本地为求交结果集合评分
17   KeepBestN(1) # 保留最佳结果
18 GroundTruth() # 与预先计算的结果比较
```

算法 3: GoT 用于 128 元素集合求交用例的配置

```
1  Generate(k=1) # 将第二个集合分成八部分, 每部分 16 个元素
2  foreach subset:
3      Generate(k=5) # 计算该子集与第一个输入集合的交集
4      Score(k=1) # 在本地为交集子集评分
5      KeepBestN(1) # 保留最佳交集子集
6  merge step 1:
7      Aggregate(5) # 合并交集子集 1 和 2
8      Score(k=1) # 在本地为求交结果集合评分
9      KeepBestN(1) # 保留最佳结果
10 merge step 2:
11     Aggregate(5) # 合并交集子集 3 和 4
12     Score(k=1) # 在本地为求交结果集合评分
13     KeepBestN(1) # 保留最佳结果
14 merge step 3:
15     Aggregate(5) # 合并交集子集 5 和 6
16     Score(k=1) # 在本地为求交结果集合评分
17     KeepBestN(1) # 保留最佳结果
18 merge step 4:
19     Aggregate(5) # 合并交集子集 7 和 8
20     Score(k=1) # 在本地为求交结果集合评分
```

算法 4: GoT 用于 128 元素集合求交用例的配置 (续)

```
21     KeepBestN(1) # 保留最佳结果
22 merge step 5:
23     Aggregate(5) # 合并步骤 1 和 2 产生的中间交集子集
24     Score(k=1) # 在本地为求交结果集合评分
25     KeepBestN(1) # 保留最佳结果
26 merge step 6:
27     Aggregate(5) # 合并步骤 3 和 4 产生的中间交集子集
28     Score(k=1) # 在本地为求交结果集合评分
29     KeepBestN(1) # 保留最佳结果
30 final merge:
31     Aggregate(5) # 合并步骤 5 和 6 产生的中间交集子集
32     Score(k=1) # 在本地为求交结果集合评分
33     KeepBestN(1) # 保留最佳结果
34 GroundTruth() # 与预先计算的结果比较
```

算法 5: GoT 用于 32 元素排序用例的配置

```
1  Generate(k=1) # 将列表分成两半, 每半 16 个元素
2  foreach list part:
3      Generate(k=5) # 对该部分列表排序
4      Score(k=1): # 为部分排序后的列表评分
5      KeepBestN(1): # 保留最佳的部分排序列表
6  Aggregate(10) # 合并两个部分排序列表
7  Score(k=1) # 在本地为排序结果列表评分
8  KeepBestN(1) # 保留最佳结果
9  Generate(k=10) # 尝试改进解
10 Score(k=1) # 在本地为排序结果列表评分
11 KeepBestN(1) # 保留最佳结果
12 GroundTruth() # 与预先计算的结果比较
```

算法 6: GoT 用于 64 元素排序用例的配置

```
1  Generate(k=1) # 将列表分成四部分，每部分 16 个元素
2  foreach list part:
3      Generate(k=5) # 对该部分列表排序
4      Score(k=1) # 为部分排序后的列表评分
5      KeepBestN(1) # 保留最佳的部分排序列表
6  merge step 1:
7      Aggregate(10) # 合并部分排序列表 1 和 2
8      Score(k=1) # 在本地为部分排序结果列表评分
9      KeepBestN(1) # 保留最佳结果
10     Generate(k=5) # 尝试改进部分解
11     Score(k=1) # 在本地为部分排序结果列表评分
12     KeepBestN(1) # 保留最佳结果
13 merge step 2:
14     Aggregate(10) # 合并部分排序列表 3 和 4
15     Score(k=1) # 在本地为部分排序结果列表评分
16     KeepBestN(1) # 保留最佳结果
17     Generate(k=5) # 尝试改进部分解
18     Score(k=1) # 在本地为部分排序结果列表评分
19     KeepBestN(1) # 保留最佳结果
20 final merge:
21     Aggregate(10) # 合并步骤 1 和 2 产生的部分排序列表
22     Score(k=1) # 在本地为排序结果列表评分
23     KeepBestN(1) # 保留最佳结果
24     Generate(k=10) # 尝试改进解
25     Score(k=1) # 在本地为排序结果列表评分
26     KeepBestN(1) # 保留最佳结果
27 GroundTruth() # 与预先计算的结果比较
```

算法 7: GoT 用于 128 元素排序用例的配置

```
1  Generate(k=1) # 将列表分成八部分, 每部分 16 个元素
2  foreach list part:
3      Generate(k=5) # 对该部分列表排序
4      Score(k=1) # 为部分排序后的列表评分
5      KeepBestN(1) # 保留最佳的部分排序列表
6  merge step 1:
7      Aggregate(10) # 合并部分排序列表 1 和 2
8      Score(k=1) # 在本地为部分排序结果列表评分
9      KeepBestN(1) # 保留最佳结果
10     Generate(k=5) # 尝试改进部分解
11     Score(k=1) # 在本地为部分排序结果列表评分
12     KeepBestN(1) # 保留最佳结果
13 merge step 2:
14     Aggregate(10) # 合并部分排序列表 3 和 4
15     Score(k=1) # 在本地为部分排序结果列表评分
16     KeepBestN(1) # 保留最佳结果
17     Generate(k=5) # 尝试改进部分解
18     Score(k=1) # 在本地为部分排序结果列表评分
19     KeepBestN(1) # 保留最佳结果
20 merge step 3:
21     Aggregate(10) # 合并部分排序列表 5 和 6
22     Score(k=1) # 在本地为部分排序结果列表评分
23     KeepBestN(1) # 保留最佳结果
24     Generate(k=5) # 尝试改进部分解
25     Score(k=1) # 在本地为部分排序结果列表评分
26     KeepBestN(1) # 保留最佳结果
27 merge step 4:
28     Aggregate(10) # 合并部分排序列表 7 和 8
29     Score(k=1) # 在本地为部分排序结果列表评分
30     KeepBestN(1) # 保留最佳结果
31     Generate(k=5) # 尝试改进部分解
32     Score(k=1) # 在本地为部分排序结果列表评分
33     KeepBestN(1) # 保留最佳结果
34 merge step 5:
35     Aggregate(10) # 合并步骤 1 和 2 产生的部分排序列表
36     Score(k=1) # 在本地为部分排序结果列表评分
37     KeepBestN(1) # 保留最佳结果
38     Generate(k=5) # 尝试改进部分解
39     Score(k=1) # 在本地为部分排序结果列表评分
40     KeepBestN(1) # 保留最佳结果
41 merge step 6:
42     Aggregate(10) # 合并步骤 3 和 4 产生的部分排序列表
43     Score(k=1) # 在本地为部分排序结果列表评分
44     KeepBestN(1) # 保留最佳结果
45     Generate(k=5) # 尝试改进部分解
46     Score(k=1) # 在本地为部分排序结果列表评分
47     KeepBestN(1) # 保留最佳结果
48 final merge:
49     Aggregate(10) # 合并步骤 5 和 6 产生的部分排序列表
50     Score(k=1) # 在本地为部分排序结果列表评分
51     KeepBestN(1) # 保留最佳结果
52     Generate(k=10) # 尝试改进解
53     Score(k=1) # 在本地为排序结果列表评分
54     KeepBestN(1) # 保留最佳结果
55 GroundTruth() # 与预先计算的结果比较
```

译者补充：图中英文对照

以下对照翻译原图内的英文。原图的数值、颜色和连接结构予以保留，API 名及提示标签保持原拼写。

图 1：提示方案

Input: 输入; Output: 输出; Thoughts: 思考; Unscored: 未评分; Negative score: 负分; Positive score: 正分; Dependencies between thoughts: 思考之间的依赖; Abandon thought: 放弃思考; Backtrack / Backtracking: 回溯; Legend: 图例; This work: 本文。Basic Input-Output (IO): 基础输入—输出; Chain-of-Thought (CoT): 思维链; Multiple CoTs (CoT-SC): 多条思维链; Tree of Thoughts (ToT): 思维树; Graph of Thoughts (GoT): 思维图。CoT 的关键创新是在一条链内产生 LLM 中间思考。CoT-SC 相对 CoT 的关键创新是利用多条独立思维链, 包括从一条链分支、选择分数最高的链、放弃某条链、聚合多条链及从链回溯。ToT 相对 CoT-SC 的关键创新是基于任意给定思考生成多个新思考, 进一步探索, 并可能从该思考回溯; 中间思考也会被评分。GoT 相对 ToT 的关键创新是任意的图式思考变换, 包括把多个思考聚合成一个新思考, 以及围绕同一思考形成循环以反复精化。Refining: 精化; Aggregating thoughts: 聚合思考。

图 2：思考变换

Graph theory view: 图论视角; Example sorting task: 排序任务示例; Example writing task: 写作任务示例; Aggregation: 聚合; Generation: 生成; Article 1 / 2 / 3: 文章 1 / 2 / 3; Keyword summary: 关键词摘要; Keyword summary 1 / 2: 关键词摘要 1 / 2。一个顶点建模一个思考, 一条边建模依赖。排序中的聚合把有序数组组合并为有序数字数组, 生成则把未排序数组拆成供后续排序的子数组。写作中的聚合把多篇文章组合为连贯摘要, 生成则从一篇文章生成多个摘要, 以尽可能提高质量。

图 3：架构、API 与完整提示文本

架构和图例。 Architecture overview: 架构概览; User: 用户; Controller: 控制器, 其目标是启动、协调、管理并推进 GoT 执行; Graph of Operations: 操作图, 目标是规定 LLM 思考变换; Graph Reasoning State: 图推理状态, 目标是维护正在进行的 LLM 推理过程; Prompter: 提示器, 目标是构造发送给 LLM 的提示; Parser: 解析器, 目标是从 LLM 思考中提取信息; Scoring & validation: 评分与验证, 目标是评估 LLM 解的质量; Ranking: 排序, 目标是指出分数最高的思考; Human or LLM: 人工或 LLM。Gray block: 灰色块, 外部实体; Blue block: 蓝色块, GoT 框架模块; Prompt: 提示; Thought: 思考; Thought state: 思考状态; Score: 分数; Operation: 操作; Dependency: 依赖; Thought state + its associated operations: 思考状态及其关联操作; Thought state + thought's score: 思考状态及该思考的分数。Specifying the Structure of the Graph of Operations: 指定操作图结构。操作图既能方便地规定 GoT, 也能规定 CoT、CoT-SC、ToT 等已有方案。Example prompts and the Graph Reasoning State for the sorting use case: 排序用例的提示及图推理状态示例。因空间所限, 每个提示中的部分示例被省略。The input thought t / thoughts t_1, t_2 : 输入思考 t / 输入思考 t_1, t_2 ; A prompt used by: 供所列操作使用的提示。

控制器 API。 参数包括: 所用 LLM 模型、温度、最大 Token 数、API 密钥、组织等; 提示 Token 成本和响应 Token 成本等 LLM 成本项; Prompter、Parser 及 Graph of Operations 的实例; 其他输入参数, 例如待排序数字。构建 GoO 时可用的操作 (可扩展) 包括 Generate、Aggregate、Score 等, 见提示器 API; KeepBest(N) 保留分数最高的 N 个思考; Repeat(k) 将给定操作重复 k 次, 生成 k 个思考。例如, 它使 Aggregate 生成多个合并结果; 每个结果作为独立思考维护在图推理状态中, 并分别评分。

提示器与解析器 API (均可扩展)。 Generate(t, k): 利用思考 t , 为 k 个新思考生成提示; ValidateAndImprove(t): 生成用于增强思考 t 的提示; Aggregate($t_1, \dots, t_k$): 生成用于组合思考 t_1 至 t_k 的提示; Score(t): 为思考 t 评分; Validate(t): 生成用于验证思考 t 正确性的提示。解析器接口包括 ParseGenerate、ParseImprove、ParseScore、ParseAggregate、ParseValidate 等。每个过程负责解析与相应提示器过程对应的 LLM 思考, 例如 ParseScore 解析 Score 的输出。

初始/系统提示 (可选)。 你好。我想对以下输入数字序列排序: {input}。

示例 1: Generate($t, k=4$)。 <Instruction> 将以下含 64 个数字的列表拆成 4 个各含 16 个数字的列表。第一个列表应包含前 16 个数字, 第二个包含第二组 16 个数字, 第三个包含第三组 16 个数字, 第四个包含第四组 16 个数字。只按以下格式输出最终的 4 个列表, 不要输出额外文字或思考!

```
{{
  "List 1": [3, 4, 3, 5, 7, 8, 1, ...],
  "List 2": [2, 9, 2, 4, 7, 1, 5, ...],
  "List 3": [6, 9, 8, 1, 9, 2, 4, ...],
  "List 4": [9, 0, 7, 6, 5, 6, 6, ...]
}}
```

</Instruction><Example> 输入:

```
[3, 1, 9, 3, 7, 5, 5, 4, 8, 1, 5, 3, 3, 2, 3, 0, 9, 7, 2, 2, 4, 4, 8, 5, 0, 8, 7, 3, 3, 8, 7, 0, 9, 5, 1, 6, 7, 6, 8, 9, 0, 3, 0, 6, 3, 4, 8, 0, 6, 9,
↪ 8, 4, 1, 2, 9, 0, 4, 8, 8, 9, 9, 8, 5, 9]
```

输出:

```
{{
  "List 1": [3, 1, 9, 3, 7, 5, 5, 4, 8, 1, 5, 3, 3, 2, 3, 0],
  "List 2": [9, 7, 2, 2, 4, 4, 8, 5, 0, 8, 7, 3, 3, 8, 7, 0],
  "List 3": [9, 5, 1, 6, 7, 6, 8, 9, 0, 3, 0, 6, 3, 4, 8, 0],
  "List 4": [6, 9, 8, 4, 1, 2, 9, 0, 4, 8, 8, 9, 9, 8, 5, 9]
}}
```

</Example> 输入: {input}。该提示用于分支因子为 $k=4$ 的 Generate 操作。根据 LLM 对此提示的回复构造四个新思考。

示例 2: Generate($t, k=1$)+Repeat($k=4$)。 <Instruction> 将以下数字列表按升序排序。只输出排序后的数字列表, 不要输出额外文字。</Instruction><Example> 输入:

[3, 7, 0, 2, 8, 1, 2, 2, 2, 4, 7, 8, 5, 5, 3, 9, 4, 3, 5, 6, 6, 4, 4, 5, 2, 0, 9, 3, 3, 9, 2, 1]

输出:

[0, 0, 1, 1, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4, 5, 5, 5, 5, 6, 6, 7, 7, 8, 8, 9, 9, 9]

</Example> 输入: {input}。该提示用于分支因子 $k=1$ 的 Generate 操作, 即只生成一个思考。但将其与 $k=4$ 的 Repeat 串接后, 底层 GoT 框架会确保 Generate 执行 4 次, 得到四个独立思考。从图论视角看, 其 GRS 与 Generate($t, k=4$) 的 GRS 相同。二者的区别是, Generate($t, k=4$) 使用户可以更灵活地控制多个思考的构造方式; Generate($t, k=1$)+Repeat($k=4$) 灵活性较低, 但更易使用。此外, Repeat 对相同提示获得四个上下文隔离的 LLM 响应; 不使用 Repeat 时, 四个思考在同一个上下文中生成, 必须在单次提示/会话中显式处理。

示例 3: Aggregate(t_1, t_2)+Repeat($k=3$)+KeepBest($N=1$)。 <Instruction> 采用归并排序式的方法, 将以下两个长度均为 {length1} 的有序列表合并为一个长度为 {length2} 的有序列表。只输出最终合并后的列表, 不要输出额外文字或思考! </Instruction><Approach> 按以下步骤以归并排序式的方法合并两个列表:

1. 比较两个列表的第一个元素。
2. 将较小元素追加到合并列表, 并将该元素所在列表的当前位置移到下一个元素。
3. 重复步骤 1 和 2, 直到其中一个列表为空。
4. 将非空列表的剩余元素追加到合并列表。

</Approach> 将以下两个列表合并成一个有序列表: 1: {input1}; 2: {input2}。合并后的列表: 该提示用于聚合因子 $k=2$ 的 Aggregate 操作, 即聚合两个输入思考 t_1 和 t_2 。GoT 重复该操作 3 次, 以尽可能提高质量, 最后选择最佳结果。这个示例的提示只明确要求执行合并, 其余操作均由 GoO 指定, 并由底层 GoT 框架处理。

示例 4: Improve(t)+Repeat($k=4$)。 <Instruction> 以下两个列表分别表示一个未排序的数字列表和其排序后的版本。排序后的版本有误, 请修正使其正确。确保输出列表按升序排列, 元素个数与输入列表相同 ({length}), 并且所含元素与输入列表相同。</Instruction><Approach> 按以下步骤修正排序错误的列表:

1. 对 0 到 9 的每个数字, 比较它在排序错误列表与输入列表中的出现次数。
2. 遍历排序错误的列表, 按需添加或删除数字, 使每个数字在该列表中的出现次数与输入列表一致。

</Approach><Examples> 输入:

[3, 7, 0, 2, 8, 1, 2, 2, 2, 4, 7, 8, 5, 5, 3, 9]

排序错误的列表:

[0, 0, 0, 0, 0, 1, 2, 2, 3, 3, 4, 4, 4, 5, 5, 7, 7, 8, 8, 9, 9, 9, 9]

原因: 排序错误的列表多出四个 0、两个 4 和三个 9, 并且少了两个 2。输出:

[0, 1, 2, 2, 2, 2, 3, 3, 4, 5, 5, 7, 7, 8, 8, 9]

输入:

[6, 4, 5, 7, 5, 6, 9, 7, 6, 9, 4, 6, 9, 8, 1, 9, 2, 4, 9, 0, 7, 6, 5, 6, 6, 2, 8, 3, 9, 5, 6, 1]

排序错误的列表:

[0, 1, 1, 2, 2, 3, 4, 4, 4, 4, 4, 5, 5, 5, 5, 6, 6, 6, 6, 6, 6, 7, 7, 7, 8, 8, 9, 9, 9, 9, 9]

原因: 排序错误的列表多出两个 4, 并且少了两个 6 和一个 9。输出:

[0, 1, 1, 2, 2, 3, 4, 4, 4, 5, 5, 5, 5, 6, 6, 6, 6, 6, 6, 6, 7, 7, 7, 8, 8, 9, 9, 9, 9, 9]

</Examples> 输入: {input}; 排序错误的列表: {incorrectly_sorted}。该提示用于 Improve(t) 操作, 利用另一个思考提供的信息增强给定思考 t 。根据用户在 GoT 中如何实现 Improve + Repeat, 它既可以像 Generate + Repeat 一样在 GRS 中生成多个新思考 (右侧上图), 也可以在 GRS 中反复精化同一个思考 (右侧下图), 将 $k=4$ 次精化迭代串接起来。

图 4：排序操作图

Graph of Operations for sorting 64 numbers: 64 个数字排序的操作图; Partial solution: 部分解; Sort: 排序; Score: 评分; Generate: 生成; KeepBest: 保留最佳; Aggregate: 聚合; Keep the best scored thoughts: 保留评分最高的思考; 16 / 32 / 64 numbers: 16 / 32 / 64 个数字; Input: 输入; Legend: 图例。Details of the highlighted part of GoO are below: GoO 高亮部分的详细内容见下方; Details of the highlighted part of the GoO from above: 上方 GoO 高亮部分的详细内容。首先, Generate 将含 64 个元素的输入数组拆成四块, 每块 16 个元素。排序在 Generate 操作内实现; k=3 表示为每个 16 元素块生成三个不同的排序结果。Score 评估每个序列排序得如何。N=1 表示从三个输入排序结果中保留一个最佳结果。Aggregate 的 k=10 表示尝试将两个输入的 16 元素子数组聚合 10 次, 合并为 32 元素子数组, 随后合并为 64 元素数组。How do we score?: 如何评分? 为获得分数, 对数字 0 至 9 分别计算输入列表与排序后列表之间的出现次数差异, 再把这 10 个值相加。零表示排序正确。为显示为“越高越好”, 采用 $\max(\text{input_length} - \text{score}, 0)$ 。图中分数 78%、86%、100%、97% 等保留原值。注意, 这只是图分解的一个示例, 各操作之间的连接结构可以任意修改。

各评估图的坐标轴和注释

图 5: incorrectly sorted elements, the lower the better: 排序错误的元素数, 越低越好; 32 / 64 / 128 elements: 32 / 64 / 128 个元素; Total Cost (\$), the lower the better: 总成本 (美元), 越低越好; clipped: 已截断; GoT: Figure 4 & Appendix: GoT 配置见图 4 及附录。图 6: incorrect elements, the lower the better: 错误元素数, 越低越好; Solved correctly: 正确求解的样本数; GoT: Appendix: GoT 配置见附录。其规模和成本标签与排序图相同。图 7: Number of errors, the lower the better: 错误数, 越低越好; Samples solved correctly: 正确求解的样本数; Total Cost (\$), the lower the better: 总成本 (美元), 越低越好。GoT4 将输入文本拆成 4 个片段, 在每个片段中统计关键词, 再始终以两个为一组聚合子结果。GoT8 与 GoT4 相同, 但拆成 8 个片段。GoTx 将输入拆成句子, 每个输入有 12 至 19 个句子。图 8: Score (out of 10), the higher the better: 评分 (满分 10 分), 越高越好; Total Cost (\$), the lower the better: 总成本 (美元), 越低越好; Aggregation of fully merged NDAs: 聚合已经完整合并的 NDA; Aggregation of partially merged NDAs: 聚合仅部分合并的 NDA。图 9 与图 10: correct elements, the higher the better: 正确元素数, 越高越好; 其规模、成本及正确求解样本数等其余标签与对应正文图相同; GoT: Figure 4: GoT 配置见图 4。

译者对示例的校核说明。 关键词计数示例保留国家名的英文拼写, 以与原输入和输出字典的键一致。协议中的方括号内容为原示例占位符, 继续保留。示例的错误输出、重复键、评分差异与代码拼写错误均忠实保留。原文在排序及集合拆分的步骤 1 响应中, 将两个键都写成了 "List 1"; 集合求交执行计划将求交尝试写作“排序尝试”。原始 Improve 示例对数字频次错误的解释未必正确, 译文并未修正模型的解释。配置中的 Aggregate(10) 拼写和 KeepBestN(1 的缺失右括号也保留原状。NDA 评分理由属于模型输出, 例如它将竞业限制归到 Doc3 / Doc4, 实际输入将其放在 Doc2; 这些描述未作为译者事实判断。